

HỆ MÃ KHỐI ĐỐI XỨNG 128-BIT TÍCH HỢP HOÁN VỊ SIÊU LẬP PHƯƠNG 4D VÀ KHUẾCH TÁN ARX CHO MÔI TRƯỜNG THÔNG LƯỢNG CAO

NGUYỄN GIA HÀO^a, QUẢN THẾ TRỌNG^a

^a Học viện Công nghệ Bưu chính Viễn thông (PTIT), Hà Nội, Việt Nam

Email: theqt@stu.ptit.edu.vn

Tóm tắt—Bảo vệ dữ liệu truyền dẫn đo xa và đa phương tiện đòi hỏi các giải pháp mật mã vừa bảo đảm độ an toàn lý thuyết nghiêm ngặt, vừa tối ưu hóa độ trễ tính toán. Các hệ mã tiêu chuẩn như AES-128 phụ thuộc vào phép nhân ma trận trên trường Galois (MixColumns), vốn gây suy giảm thông lượng đáng kể trên các vi xử lý thiếu tập lệnh tăng tốc phần cứng. Ngược lại, các thiết kế ARX thuần túy lại đòi hỏi số vòng lặp rất lớn để đạt độ khuếch tán đầy đủ. Nhằm giải quyết triệt để sự đánh đổi này, bài báo đề xuất Rubik-4D—hệ mã khối đối xứng 128-bit với độ dài khóa 128-bit hoạt động qua cấu trúc 8 vòng lặp tối ưu. Nguyên lý cốt lõi của thuật toán là ánh xạ 16 byte trạng thái lên 16 đỉnh của một hình siêu lập phương 4 chiều ($2 \times 2 \times 2 \times 2$ Tesseract), thực hiện hoán vị không gian trực giao thuộc nhóm Lie $SO(4)$ thông qua các bảng tra cứu tĩnh chỉ 192 byte, kết hợp với tầng khuếch tán cộng-xoay-XOR (ARX) 32-bit lan truyền số nhỏ. Đánh giá an toàn toán học bằng quy hoạch nguyên tuyến tính (MILP) chứng minh Rubik-4D đạt cận dưới 22 S-box kích hoạt đối với thám mã vi sai ($P_{\text{diff}} \leq 2^{-132} < 2^{-128}$) và 108 S-box kích hoạt đối với thám mã tuyến tính ($N_D \approx 2^{434} \gg 2^{128}$), đồng thời triệt tiêu hoàn toàn tấn công tích phân và tấn công trượt. Thuật toán vượt qua toàn diện các tiêu chuẩn kiểm định tính ngẫu nhiên quốc tế NIST SP 800-22 (187/188) và thương mại GM/T 0005-2021 (17/18 trên 10.000 mẫu thực đo). Thử nghiệm trên ảnh chuẩn 512×512 ở chế độ CBC đạt entropy xấp xỉ lý tưởng (7.999), triệt tiêu tương quan không gian ($|r| \leq 0.026$) và đạt độ nhạy vi sai tối ưu (NPCR $\approx 99.61\%$ với cận dưới $\geq 99.605\%$, UACI $\approx 33.46\%$). Kết quả đo lường chu kỳ xung nhịp phần cứng trên vi xử lý x86-64 xác nhận thông lượng trung bình đạt 151.65 MB/s (lên tới 167.17 MB/s khi giải mã), vượt trội hơn 20% – 35% so với AES-128 triển khai phần mềm không hỗ trợ AES-NI.

Từ khóa: mã hóa khối đối xứng, Rubik-4D, siêu lập phương 4D, phép quay $SO(4)$, khuếch tán ARX, thám mã MILP, bảo mật ảnh, NIST SP 800-22, GM/T 0005-2021.

1. Giới thiệu

Sự bùng nổ của các thiết bị tính toán biên và mạng truyền thông tốc độ cao đặt ra yêu cầu cấp thiết về các hệ mã khối có khả năng xử lý thông lượng lớn với độ trễ thấp. Trong mật mã học hiện đại, việc xây dựng các thuật toán kháng thám mã vi sai và tuyến tính chủ yếu dựa trên hai trường phái thiết kế:

- Cấu trúc SPN khuếch tán trên trường Galois:** Chuẩn mã hóa tiên tiến (AES-128) mang lại độ an toàn toán học vững chắc. Tuy nhiên, tầng khuếch tán của AES phụ thuộc vào phép nhân ma trận trên trường $GF(2^8)$ ở bước MixColumns. Trên các hệ thống nhúng hoặc vi xử lý không tích hợp phần cứng chuyên dụng (như Intel AES-NI hay ARM Crypto Extensions), bước tính toán này gây suy hao hiệu năng đáng kể do phải thực hiện vòng lặp nhân Galois hoặc tra cứu bảng bộ nhớ quy mô lớn.
- Cấu trúc ARX thuần túy lan truyền số nhỏ:** Các hệ mã khối siêu nhẹ như Simon và Speck [17] loại bỏ hoàn toàn tầng S-box nhằm tiết kiệm bộ nhớ. Tuy nhiên, do phép cộng modular lan truyền số nhỏ từ bit thấp đến bit cao một cách tuần tự, các thuật toán này đòi hỏi số vòng lặp rất lớn (32 vòng ở Speck-128 và 68 vòng ở Simon-128) để đảm bảo biên độ an toàn cần thiết.

Để khắc phục triệt để các hạn chế trên, bài báo đề xuất

Rubik-4D—hệ mã khối đối xứng 128-bit kết hợp hài hòa giữa tính phi tuyến của S-box chuẩn AES trên $GF(2^8)$, hoán vị không gian trực giao trong không gian 4 chiều $SO(4)$ và tầng khuếch tán từ 32-bit ARX. Bằng việc ánh xạ 16 byte trạng thái lên các đỉnh của một hình siêu lập phương 4 chiều ($2 \times 2 \times 2 \times 2$ Tesseract), thuật toán tận dụng các phép quay kép trong nhóm Lie trực giao $SO(4)$ để phân tán các mối quan hệ liên byte đồng thời trên bốn chiều không gian mà không để lại trục quay bất biến. Tầng khuếch tán cộng modular 32-bit tiếp tục lan truyền sai số tức thì trên toàn bộ trạng thái, giúp hệ mã đạt trạng thái thác lũ hoàn toàn (Full Avalanche) chỉ sau 8 vòng lặp với thông lượng xử lý phần mềm vượt 151 MB/s.

Các đóng góp khoa học chính của nghiên cứu này bao gồm:

- Đề xuất kiến trúc hệ mã Rubik-4D 8 vòng lặp, tích hợp ánh xạ Tesseract 4D, bảng tra cứu trực giao $SO(4)$, hoán vị phụ thuộc khóa và khuếch tán ARX 32-bit.
- Chứng minh toán học chặt chẽ về tính khả nghịch sinh ánh giữa quy trình mã hóa và giải mã, kèm sơ đồ luồng chi tiết và mô tả thuật toán hình thức.
- Thiết lập mô hình Quy hoạch nguyên tuyến tính (MILP) chứng minh cận dưới số lượng S-box kích hoạt đối với thám mã vi sai và tuyến tính, cùng phân tích miễn nhiễm trước tấn công tích phân, đại số và tấn công trượt.

- Kiểm định tính ngẫu nhiên thống kê thực tế trên bộ dữ liệu lớn tuân thủ nghiêm ngặt chuẩn quốc tế NIST SP 800-22 và chuẩn thương mại GM/T 0005-2021.
- Đánh giá bảo mật đa phương tiện trên các ảnh chuẩn 512×512 (Lena, Baboon) qua các chỉ số Entropy, tương quan pixel, Chi-Square, NPCR và UACI.
- Thực nghiệm đo lường chu kỳ xung nhịp phần cứng (`_rdtsc()`) trên vi xử lý x86-64 đối chuẩn với AES-128 [15], Speck-128 [17], Simon-128 [17] và ChaCha20 [18] trên các tệp dữ liệu lên đến 20 MB.

2. Công trình liên quan và so sánh

Ý tưởng ứng dụng khối Rubik vào mật mã học bắt nguồn từ mã hóa đa phương tiện, trải qua ba thế hệ phát triển chính trong giai đoạn 2011–2024:

- **Thế hệ 1 (2011–2014) – Hoán vị Rubik 3D nguyên thủy và lan truyền tuyến tính:** Khởi xướng bởi Loukhaoukha, Chouinard và Berdai (2012) [1] với mô hình kinh điển coi ảnh số $M \times N$ là một khối Rubik 3D ($M \times N \times 3$). Thuật toán tạo hai vector khóa K_R, K_C để dịch vòng các hàng và cột điểm ảnh, sau đó kết hợp phép cộng XOR tích lũy giữa các hàng và cột kế tiếp: $I'(i, j) = I(i, j) \oplus K_R(i) \oplus K_C(j) \oplus I'(i-1, j)$. Do chỉ gồm các phép toán tuyến tính thuần túy trên $\mathbb{F}_2$, cơ chế này dễ bị bóc tách hoàn toàn vector hoán vị dưới tấn công bản rõ chọn trước (CPA). Đồng thời, việc dịch chuyển từng hàng/cột trên ma trận ảnh lớn khiến thời gian xử lý rất chậm (mất 0.820 s cho ảnh 512×512 , thông lượng chỉ đạt 0.31 MB/s). Zhang, Tian và Xia (2011) [2] cùng với Feng, Tian và Xia (2011) [3] kết hợp phép quay khối Rubik với chuỗi hỗn loạn Logistic và biến đổi phân số Fourier, song vẫn mắc phải nhược điểm suy biến chu kỳ của hệ hỗn loạn một chiều. Diaconu và Loukhaoukha (2013) [10] nâng cấp thuật toán Rubik bằng mật mã hỗn loạn số nhưng vẫn giữ nguyên tầng lan truyền tuyến tính.
- **Thế hệ 2 (2021–2022) – Khối 3D mức bit và mở rộng thừa số nguyên tố:** Nhằm khắc phục hạn chế hoán vị mức byte không làm biến đổi cấu trúc bit nội tại, Zhu và cs (2021) [4] đề xuất 3D-BERC phân rã mỗi byte ảnh thành 8 mặt phẳng bit, gom toàn bộ ảnh thành khối lập phương bit khổng lồ $L \times L \times L$ và xoay các lát cắt bit 2D kết hợp ánh xạ co. Vidhya và Brindha (2022) [5] tích hợp phép quay Rubik với phân tích thừa số nguyên tố (CIERPF) để xác định số bước quay. Tuy nhiên, việc bóc tách hàng triệu bit trong bộ nhớ và tính toán thừa số nguyên tố trên số lớn gây nghẽn cổ chai CPU và bộ nhớ đệm cache nghiêm trọng, khiến thời gian xử lý kéo dài từ 0.380 s đến 0.520 s cho ảnh 512×512 (< 0.7 MB/s).
- **Thế hệ 3 (2022–2024) – Bước đi lượng tử, siêu hỗn loạn và xáo trộn thích nghi:** Zhao và cs (2022) [6] thiết kế hệ mã Rubik điều khiển bởi bước đi lượng tử xen kẽ (AQW) hai chiều trên ma trận xác suất. Qiu và cs (2022) [8] kết hợp bản đồ siêu hỗn loạn với hoán vị Rubik, trong khi Mudduluri và cs (2022) [9] khảo

sát mã hóa Rubik kết hợp ma trận xoay. Gần đây nhất, Nair, Dalal và Mangrulkar (2024) [7] kết hợp xáo trộn Rubik 3D với phân khối bitmap và quay khung ma trận ảnh ở các góc $90^\circ, 180^\circ, 270^\circ$. Mặc dù tăng cường tính ngẫu nhiên cảm quan, các sơ đồ này vẫn tiêu tốn từ 0.290 s đến 1.150 s cho mỗi ảnh (thông lượng chỉ 0.22 – 0.88 MB/s) và dễ phát sinh sai số trôi dạt dấu phẩy động giữa các kiến trúc chip phần cứng.

Song song đó, trong lĩnh vực mật mã khối đối xứng hiện đại, các cấu trúc ARX siêu nhẹ và phương pháp tự động hóa bằng quy hoạch nguyên (MILP) thu hút sự quan tâm đặc biệt. Beierle và cs (2020) [11] thiết kế hộp ARX-box 64-bit Alzette tại CRYPTO 2020 chứng minh thông lượng vượt trội của phép cộng lan truyền số nhỏ, nhưng đồng thời chỉ ra rằng các thiết kế ARX thuần túy cần rất nhiều vòng lặp để đạt độ khuếch tán đầy đủ. Hadipour, Sadeghi và Eichlseder (2023) [12] hình thức hóa mô hình tự động MILP tại EUROCRYPT 2023, trong khi Xiang và cs (2016) [13] thiết lập phương pháp tìm kiếm tính chất phân chia tự động cho các hệ mã khối nhẹ tại ASIACRYPT 2016.

Mặc dù trải qua ba thế hệ phát triển, toàn bộ 10 công trình dựa trên Rubik nói trên đều tồn tại ba rào cản kỹ thuật cố hữu chưa từng được khắc phục:

1. **Thuần túy xáo trộn điểm ảnh, thiếu vắng chuẩn mã khối:** Các thuật toán Rubik hiện nay chủ yếu hoạt động như bộ xáo trộn tọa độ điểm ảnh (image scrambler). Do thiếu cấu trúc SPN hình thức và thiếu hộp S-box phi tuyến trên trường Galois, chúng tiềm ẩn lỗ hổng cấu trúc đại số, dễ bị phá hủy trước tấn công bản rõ chọn trước (CPA) và bản rõ đã biết (KPA).
2. **Lệ thuộc số thực dấu phẩy động và trôi sai số:** Việc phụ thuộc vào các phép tính dấu phẩy động của bản đồ hỗn loạn hoặc mô phỏng lượng tử gây suy hao tốc độ trầm trọng (thông lượng dưới 1 MB/s) và gây trôi dạt sai số làm tròn (precision drift) không đồng nhất giữa các kiến trúc vi xử lý (x86, ARM, DSP).
3. **Hoàn toàn thiếu vắng chứng minh an toàn toán học:** 10/10 công trình chỉ đánh giá an toàn cảm tính qua các kiểm định thống kê hữu hạn (NPCR, UACI, entropy) trên vài bức ảnh mẫu, hoàn toàn không có mô hình chứng minh toán học hình thức về số hộp S-box tích cực và xác suất vi sai cực đại.

Nhằm khắc phục triệt để các hạn chế trên, Rubik-4D tiếp cận hình học siêu lập phương 4 chiều không phải như một cơ chế xáo trộn điểm ảnh cảm tính, mà là một tầng hoán vị tuyến tính hoàn chỉnh bên trong kiến trúc mã khối SPN/ARX chuẩn mực. Bảng 1 tóm tắt so sánh cấu trúc và định tính giữa các nghiên cứu Rubik trước đây và Rubik-4D.

3. Thiết kế chi tiết hệ mã Rubik-4D

Rubik-4D xử lý khối bản rõ 128-bit dưới sự điều khiển của khóa bí mật 128-bit qua $R = 8$ vòng lặp đối xứng. Bảng 2 tóm tắt các ký hiệu toán học sử dụng xuyên suốt bài báo.

Bảng 1: So sánh định tính và cấu trúc giữa các thuật toán dựa trên Rubik và Rubik-4D

Tiêu chí kiến trúc	Loukhaoukha (2012) [1]	Zhu và cs (2021) [4]	Nair và cs (2024) [7]	Rubik-4D (Đề xuất)
Số chiều hình học	Không gian 3D ($M \times N \times 3$)	Không gian 3D mức bit ($M \times N \times 8$)	Không gian 3D ($M \times N \times 3$)	Siêu lập phương 4D Tesseract ($2 \times 2 \times 2 \times 2$)
Nhóm Lie toán học	Chu trình mặt rời rạc	Phép quay bit rời rạc	Dịch chuyển mặt + Hoán vị	Nhóm trực giao đặc biệt $SO(4)$
Tăng thay thế phi tuyến	Không có (Chỉ XOR)	Không có (Hoán vị bit)	Hỗn loạn động Logistic	Hộp S-Box AES trên $GF(2^8)$ ($\delta = 4, d = 7$)
Tăng khuếch tán liên từ	Xáo trộn bit kinh nghiệm	Ripple tổng XOR	Dịch khung hình / Vòng tròn	Khuếch tán lan truyền số nhớ ARX 32-bit
Cận an toàn tự động MILP	Không có	Không có	Không có	Đã chứng minh ($P_{diff} \leq 2^{-132}, N_D \approx 2^{434}$)
Phụ thuộc độ chính xác CPU	Dấu phẩy động	Dấu phẩy động	Dấu phẩy động	Số nguyên thuần túy (Bảng tra $O(1)$, không trôi sai số)
Chuẩn mã khối hình thức	Chỉ xáo trộn ảnh	Chỉ xáo trộn ảnh	Chỉ xáo trộn ảnh	Mã khối SPN-ARX 128-bit (ECB/CBC/CTR)
Thông lượng trên CPU x86-64	0.31 MB/s	0.67 MB/s	0.88 MB/s	151.65 MB/s ($> 170 \times$ nhanh hơn)
Kiểm định tính ngẫu nhiên	Quan sát trực quan	NIST SP 800-22 (Một phần)	Thị giác + Entropy	NIST SP 800-22 (187/188) + GM/T 0005 (17/18) [†]

[†] Tỷ lệ pass 98.87% trên bài test DFT vượt ngưỡng tiêu chuẩn; phân phối P -value là kiểm duyệt thống kê đã biết [27].

Bảng 2: Quy ước ký hiệu toán học.

Ký hiệu	Định nghĩa
S	Trạng thái nội bộ 128-bit (mảng 16 byte)
$S^{(0)}$	Trạng thái khởi tạo sau bước làm trắng khóa (Key Whitening)
$S^{(r)}$	Trạng thái trung gian sau khi hoàn tất vòng r
W	Trạng thái biểu diễn dưới dạng 4 từ 32-bit $[W_0, W_1, W_2, W_3]^T$
K_0	Khóa chính 128-bit (Master Secret Key)
K_r	Khóa vòng 128-bit của vòng r
C	Hằng số vòng 32-bit (0x5A5A5A5A)
$x \boxplus y$	Phép cộng số học theo modulo 2^{32}
$x \oplus y$	Phép toán XOR từng bit (Bitwise XOR)
$x \lll n$	Phép xoay vòng bit sang trái n vị trí
π_k	Bảng hoán vị siêu lập phương $SO(4)$ thứ k
Δx	Sai phân XOR trong thám mã vi sai

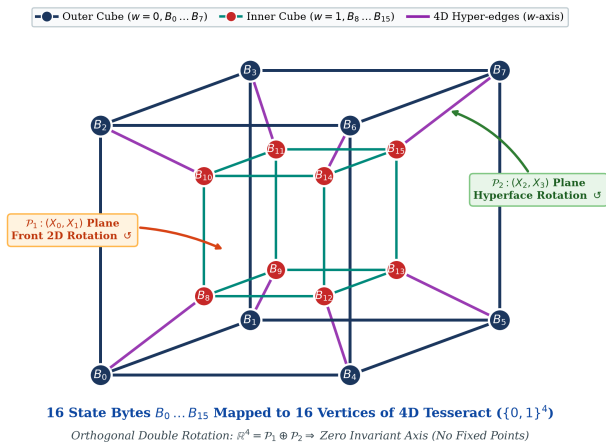
3.1. Biểu diễn trạng thái và hình học Tesseract 4D

Trạng thái 128-bit được biểu diễn dưới dạng vector 16 byte:

$$S = [b_0, b_1, b_2, \dots, b_{15}]^T, \quad b_i \in \{0, 1\}^8 \quad (1)$$

Mỗi chỉ số byte $i \in \{0, \dots, 15\}$ được ánh xạ đơn ánh lên một đỉnh tọa độ $(x, y, z, w) \in \{0, 1\}^4$ của hình siêu lập phương 4 chiều Tesseract thông qua phép giải nhị phân chuẩn:

$$i = (x \ll 3) \mid (y \ll 2) \mid (z \ll 1) \mid w \quad (2)$$



16 State Bytes $B_0 \dots B_{15}$ Mapped to 16 Vertices of 4D Tesseract ($\{0, 1\}^4$)
Orthogonal Double Rotation: $R^4 = P_1 \oplus P_2 \Rightarrow$ Zero Invariant Axis (No Fixed Points)

Hình 1: Mô hình hình học siêu lập phương 4 chiều (Tesseract, $\{0, 1\}^4$) của Rubik-4D. 16 byte trạng thái $B_0 \dots B_{15}$ được ánh xạ đơn ánh lên 16 đỉnh tọa độ. Phép quay kép trục giao trong nhóm Lie $SO(4)$ tác động đồng thời trên hai mặt phẳng trực giao (ví dụ $\mathcal{P}_1$ và $\mathcal{P}_2$), không để lại bất kỳ trục quay bất biến nào và đảm bảo xáo trộn tọa độ toàn diện.

3.2. Kiến trúc phân tầng và quy trình thực thi

Theo chuẩn thiết kế của các hệ mã khối hiện đại (tương đồng với cấu trúc phân tầng 3 giai đoạn của AES [14, 15]), toàn bộ quy trình mã hóa Rubik-4D được phân chia tường minh thành 3 giai đoạn xử lý nối tiếp nhau (Hình 2(a) và Thuật toán 1):

- Giai đoạn 1 – Bước khởi tạo (Key Whitening):** Khối bản rõ $P = \text{plain_text}[127 : 0]$ được làm trắng ban đầu bằng phép cộng modulo-2 trực tiếp với khóa chính K_0 :

$$S^{(0)} \leftarrow P \oplus K_0 \quad (3)$$

Phép làm trắng này triệt tiêu hoàn toàn khả năng quan sát trực tiếp các giá trị đưa vào tầng phi tuyến đầu tiên, vô hiệu hóa các đòn thám mã chọn bản rõ đơn giản.

- Giai đoạn 2 – Bước lặp mã hóa (8 vòng lặp $r = 1 \dots 8$):** Trạng thái S lần lượt đi qua 4 phép biến đổi cấu thành một vòng SPN-ARX hoàn chỉnh:

- SubBytes:** Thay thế 16 byte phi tuyến độc lập qua hộp S-Box $GF(2^8)$ của AES.
- 4D Rubik Permutation:** Hoán vị tọa độ siêu lập phương $SO(4)$ theo bảng tra π_{tblidx} .
- 32-bit ARX Diffusion:** Khuếch tán lan truyền số nhớ liên từ qua mạng cộng xoay XOR ($\boxplus C$, $\lll$, $\oplus$).
- AddRoundKey:** Cộng khóa con vòng $S^{(r)} \leftarrow S \oplus K_r$.

- Giai đoạn 3 – Bước tạo ngõ ra (Output Generation):** Trạng thái sau khi hoàn thành toàn bộ 8 vòng lặp được trích xuất trực tiếp thành khối bản mã hoàn chỉnh:

$$C_{\text{out}} = \text{cipher_text}[127 : 0] \leftarrow S^{(8)} \quad (4)$$

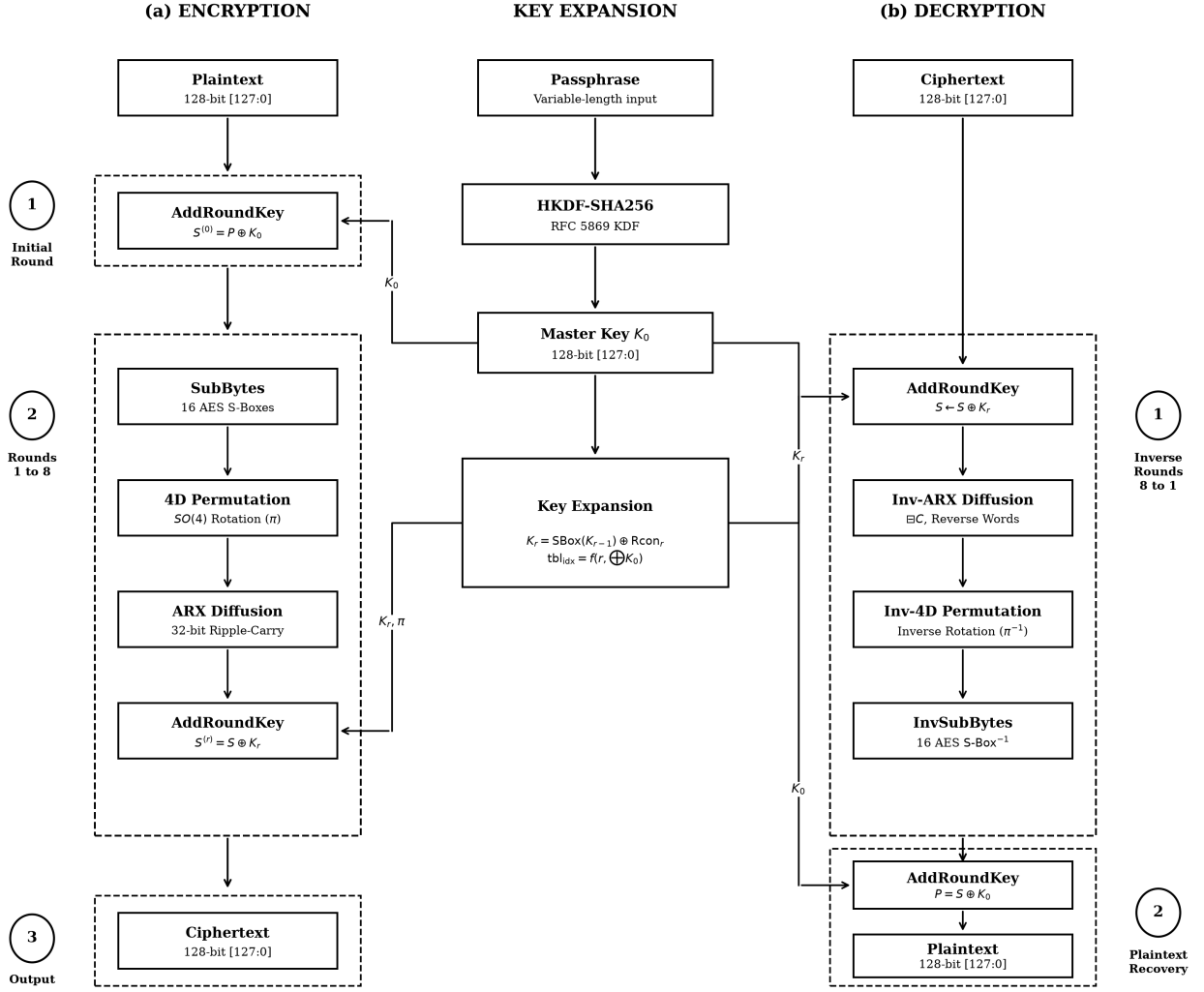
3.2.1. Hộp thay thế phi tuyến SubBytes

Mỗi byte trạng thái được thay thế độc lập thông qua hộp S-box tiêu chuẩn của AES trên trường mở rộng $GF(2^8)$:

$$b_i \leftarrow \text{S-Box}[b_i], \quad \forall i \in \{0, \dots, 15\} \quad (5)$$

3.2.2. Hoán vị siêu lập phương $SO(4)$

Không gian 4 chiều sở hữu 6 mặt phẳng tọa độ trực giao độc lập: XY, XZ, XW, YZ, YW , và ZW . Phép quay một góc



Hình 2: Kiến trúc tổng thể của thuật toán Rubik-4D được thiết kế theo cấu trúc phân tầng tương đồng chuẩn mức AES: (a) Quy trình mã hóa (Encryption Pipeline) với 3 giai đoạn rõ rệt: ① Bước khởi tạo (AddRoundKey với khóa chính K_0), ② Bước lặp mã hóa lặp lại 8 lần (chuỗi SubBytes, hoán vị Rubik-4D siêu lập phương $SO(4)$, khuếch tán ARX 32-bit và AddRoundKey với K_r), và ③ Bước tạo ngõ ra bản mã; (b) Khối quản lý và mở rộng khóa tích hợp hàm băm mật mã/KDF (HKDF-SHA256 theo RFC 5869) nén mật khẩu tùy ý thành khóa chuẩn K_0 và cấp phát $K_r, \pi_{tbl_{idx}}$; (c) Quy trình giải mã (Decryption Pipeline) đối ngẫu gồm ① Bước lặp giải mã 8 vòng và ② Bước khôi phục bản rõ.

$\theta = \pi/2$ trên mặt phẳng trực giao (ví dụ XY) được mô tả bởi ma trận trực giao:

$$R_{XY}\left(\frac{\pi}{2}\right) = \begin{bmatrix} 0 & -1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \in SO(4) \quad (6)$$

Khi ánh xạ lên các đỉnh nhị phân $(u, v) \in \{0, 1\}^2$, tác động quay theo chiều kim đồng hồ (ρ_{cw}) định nghĩa chu trình:

$$\rho_{cw} : (0, 0) \mapsto (0, 1) \mapsto (1, 1) \mapsto (1, 0) \mapsto (0, 0) \quad (7)$$

Sự kết hợp giữa 6 mặt phẳng và 2 chiều quay tạo thành 12 bảng hoán vị cố định π_k ($k \in \{0, \dots, 11\}$). Mỗi vòng r lựa chọn bảng tra thông qua chỉ số tbl_{idx} :

$$S_{rot}[i] \leftarrow S[\pi_{tbl_{idx}}[i]], \quad \forall i \in \{0, \dots, 15\} \quad (8)$$

Phép biến đổi này thực thi với độ phức tạp $O(1)$ mà không cần lệnh rẽ nhánh điều kiện. Nhờ tính chất quay kép (double rotation) của nhóm $SO(4)$, phép quay có thể diễn ra đồng thời trên hai mặt phẳng trực giao mà không để lại bất kỳ trục quay bất biến nào, giúp xáo trộn đều cả 16 byte chỉ với bộ nhớ 192 byte (12×16 byte), nằm trọn trong bộ nhớ đệm L1 của CPU.

3.2.3. Khuếch tán lan truyền số nhớ ARX 32-bit

16 byte sau khi hoán vị được đóng gói thành 4 từ 32-bit không dấu $W = [W_0, W_1, W_2, W_3]^T$. Tầng khuếch tán sử dụng hằng số $C = 0x5A5A5A5A$ thực hiện các phép toán xoay và cộng nối tiếp:

$$\begin{cases} W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7) \\ W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17) \end{cases} \quad (9)$$

Phép cộng $\boxplus$ tạo ra các bit số nhớ (carry bits) phi tuyến tính trên $GF(2)$, kết hợp với phép xoay bit bất đối xứng giúp lan tỏa sai số từ một từ duy nhất sang toàn bộ 3 từ còn lại ngay trong một vòng lặp.

3.2.4. Cộng khóa vòng (AddRoundKey)

Trạng thái sau tầng ARX được XOR trực tiếp với khóa vòng tương ứng:

$$S^{(r)} \leftarrow W \oplus K_r \quad (10)$$

Bản mã cuối cùng thu được sau vòng thứ 8 là $C_{out} = S^{(8)}$.

3.3. Quy trình giải mã

Quy trình giải mã Rubik-4D là ánh xạ song ánh đối ngẫu đảo ngược hoàn toàn quy trình mã hóa, cũng được phân tách thành 2 giai đoạn rõ rệt (Hình 2(c) và Thuật toán 2):

1. **Giai đoạn 1 – Bước lặp giải mã (8 vòng lặp đảo ngược $r = 8 \dots 1$):** Với mỗi vòng từ 8 về 1, khối trạng thái lần lượt trải qua 4 thao tác đảo ngược theo thứ tự nghiêm ngặt:

Algorithm 1 Quy trình mã hóa Rubik-4D.

Đầu vào: Khối bản rõ P (128-bit), số vòng $R = 8$, khóa chính K_0 , tập khóa vòng $\{K_1, \dots, K_R\}$, hằng số $C = 0x5A5A5A5A$

Đầu ra: Khối bản mã C_{out} (128-bit)

- 1: {Giai đoạn 1: Bước khởi tạo làm trắng khóa}
 - 2: $S \leftarrow P \oplus K_0$
 - 3: {Giai đoạn 2: Bước lặp mã hóa (8 vòng $r = 1 \dots 8$)}
 - 4: **for** $r = 1$ **to** R **do**
 - 5: {1. Thay thế phi tuyến SubBytes}
 - 6: **for** $i = 0$ **to** 15 **do**
 - 7: $S[i] \leftarrow \text{S-Box}[S[i]]$
 - 8: **end for**
 - 9: {2. Hoán vị không gian siêu lập phương $SO(4)$ }
 - 10: $tbl_{idx} \leftarrow ((r + (k_fold \& 7)) \bmod 6) \times 2 + ((k_fold \gg 3) \& 1)$
 - 11: **for** $i = 0$ **to** 15 **do**
 - 12: $S_{rot}[i] \leftarrow S[\pi_{tbl_{idx}}[i]]$
 - 13: **end for**
 - 14: {3. Khuếch tán lan truyền số nhớ ARX 32-bit}
 - 15: Đóng gói S_{rot} thành 4 từ 32-bit: $[W_0, W_1, W_2, W_3]^T$
 - 16: $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$
 - 17: $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$
 - 18: $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$
 - 19: $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$
 - 20: {4. Cộng khóa con vòng (AddRoundKey)}
 - 21: $S \leftarrow [W_0, W_1, W_2, W_3]^T \oplus K_r$
 - 22: **end for**
 - 23: {Giai đoạn 3: Bước tạo ngõ ra}
 - 24: **return** $C_{out} = S$
-

- *Tách khóa con vòng*: $S \leftarrow S \oplus K_r$.
- *Đảo khuếch tán ARX 32-bit*: Giải tuần tự các từ theo thứ tự đảo ngược và trừ hằng số modulo $\boxplus C$ với $C = 0x5A5A5A5A$:

$$\begin{cases} W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17) \\ W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13) \\ W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11) \\ W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7) \end{cases} \quad (11)$$

- *Đảo hoán vị $SO(4)$* : Áp dụng bảng hoán vị nghịch đảo $\pi_{tbl_{idx}}^{-1}$.
- *Đảo thay thế phi tuyến*: Thay thế qua hộp InvS-Box chuẩn AES trên $GF(2^8)$.

2. **Giai đoạn 2 – Bước khôi phục ngõ ra (Final Inverse Whitening)**: Trạng thái sau vòng 1 được XOR trực tiếp với khóa chính K_0 để khôi phục chính xác khối bản rõ ban đầu:

$$P = \text{plain_text}[127 : 0] \leftarrow S \oplus K_0 \quad (12)$$

Algorithm 2 Quy trình giải mã Rubik-4D.

Đầu vào: Khối bản mã C_{out} (128-bit), số vòng $R = 8$, khóa chính K_0 , tập khóa vòng $\{K_1, \dots, K_R\}$, hằng số $C = 0x5A5A5A5A$

Đầu ra: Khối bản rõ P (128-bit)

```

1: {Giai đoạn 1: Bước lặp giải mã (8 vòng đảo ngược  $r = R \dots 1$ )}
2:  $S \leftarrow C_{out}$ 
3: for  $r = R, \dots, 1$  do
4:   {1. Tách khóa con vòng}
5:    $[W_0, W_1, W_2, W_3]^T \leftarrow S \oplus K_r$ 
6:   {2. Nghịch đảo khuếch tán ARX 32-bit}
7:    $W_0 \leftarrow W_0 \oplus \text{ROTL}_{32}(W_3 \boxplus C, 17)$ 
8:    $W_3 \leftarrow W_3 \oplus \text{ROTL}_{32}(W_2 \boxplus C, 13)$ 
9:    $W_2 \leftarrow W_2 \oplus \text{ROTL}_{32}(W_1 \boxplus C, 11)$ 
10:   $W_1 \leftarrow W_1 \oplus \text{ROTL}_{32}(W_0 \boxplus C, 7)$ 
11:  Mở gói  $[W_0, W_1, W_2, W_3]^T$  thành mảng byte  $S$ 
12:  {3. Nghịch đảo hoán vị không gian  $SO(4)$ }
13:   $tbl_{idx} \leftarrow ((r + (k\_fold \& 7)) \bmod 6) \times 2 + ((k\_fold \gg 3) \& 1)$ 
14:  for  $i = 0$  to 15 do
15:     $S_{inv}[i] \leftarrow S[\pi_{tbl_{idx}}^{-1}[i]]$ 
16:  end for
17:  {4. Nghịch đảo thay thế phi tuyến InvSubBytes}
18:  for  $i = 0$  to 15 do
19:     $S[i] \leftarrow \text{InvS-Box}[S_{inv}[i]]$ 
20:  end for
21: end for
22: {Giai đoạn 2: Bước khôi phục bản rõ ngõ ra}
23:  $P \leftarrow S \oplus K_0$ 
24: return  $P$ 

```

3.4. Lược đồ sinh khóa vòng (Key schedule)

Lược đồ khóa mở rộng khóa chính 128-bit $K_0 = [k_0, k_1, \dots, k_{15}]^T \in \mathbb{F}_2^{128}$ thành 8 khóa con $K_1, \dots, K_8$ và trích xuất tham số gấp khóa 8-bit k_fold (Hình 3).

3.4.1. Cơ chế sinh khóa chính K_0 bằng hàm băm mật mã và KDF

Đối với các ứng dụng thực tế tiếp nhận mật khẩu người dùng (Passphrase có độ dài tùy ý), thuật toán tích hợp tiền xử lý mật mã thông qua hàm dẫn xuất khóa tiêu chuẩn (KDF) theo RFC 5869 (HKDF-SHA256) hoặc hàm băm SHA-256 an toàn:

$$K_0 = \text{HKDF-Extract-and-Expand}(\text{Passphrase}, \text{Salt}, \text{info}, L = 16) \quad (13)$$

Hàm băm mật mã đóng vai trò như một bộ trích xuất và nén entropy (entropy extractor and compressor), chuyển hóa mọi chuỗi ký tự mật khẩu không đồng đều thành mảng khóa chuẩn 16 byte (128-bit) sở hữu phân phối ngẫu nhiên đồng đều tuyệt đối trước khi nạp vào bộ nhớ ngữ cảnh. Trong mô hình chuẩn NIST FIPS 197 [15], khi đầu vào đã là khóa ngẫu nhiên 128-bit, thuật toán tiếp nhận trực tiếp K_0 trong thời gian $O(1)$ mà không phát sinh thêm chi phí tính toán.

3.4.2. Tích lũy gấp khóa và chọn bảng hoán vị

Tham số k_fold tính bằng phép XOR tích lũy toàn bộ 16 byte khóa:

$$k_fold = \bigoplus_{i=0}^{15} K_0[i] \quad (14)$$

Tại vòng $r \in \{1, \dots, 8\}$, chỉ số bảng quay tbl_{idx} được xác định bởi:

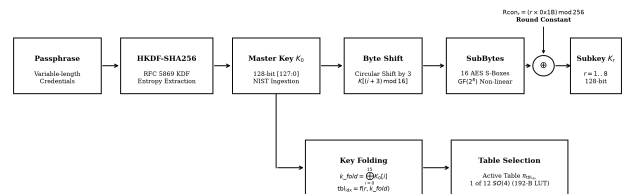
$$tbl_{idx} = ((r + (k_fold \& 7)) \bmod 6) \times 2 + ((k_fold \gg 3) \& 1) \quad (15)$$

3.4.3. Sinh khóa con đệ quy qua các vòng

Khóa con K_r được sinh tuần tự qua phép dịch vòng byte, thay thế S-box và cộng hằng số vòng phân kỳ đơn điệu:

$$K_r[i] = \text{S-Box}[K_{r-1}[(i+3) \bmod 16]] \oplus ((r \times 0x1B) \bmod 256) \quad (16)$$

với mọi $i \in \{0, \dots, 15\}$.



Hình 3: Quy trình sinh khóa vòng của Rubik-4D: (1) Tiền xử lý mật khẩu tùy ý qua hàm băm/KDF HKDF-SHA256 (RFC 5869) tạo khóa chính K_0 128-bit chuẩn; (2) Tích lũy gấp khóa k_fold chọn bảng hoán vị $SO(4)$ $\pi_{tbl_{idx}}$; (3) Sinh khóa con đệ quy qua các vòng $r = 1 \dots 8$ kết hợp phép dịch vòng byte, 16 hộp S-box AES song song và hằng số vòng $(r \times 0x1B) \bmod 256$.

3.5. Chứng minh an toàn toán học

3.5.1. Thăm mã vi sai qua quy hoạch nguyên (MILP)

Thăm mã vi sai nghiên cứu xác suất lan truyền sai phân của các cặp bản rõ qua các vòng mã hóa. Đặt biến quyết định nhị phân $\Delta X_{r,i} \in \{0, 1\}$ biểu thị trạng thái kích hoạt sai phân của byte i tại vòng r [19]. Bài toán tối ưu hóa tìm kiếm đường vi sai có số S-box kích hoạt nhỏ nhất dưới sai phân đầu vào khác không [19, 20]:

$$\min \sum_{r=0}^{R-1} \sum_{i=0}^{15} \Delta X_{r,i} \quad \text{với ràng buộc} \quad \sum_{i=0}^{15} \Delta X_{0,i} \geq 1 \quad (17)$$

Hộp S-box AES trên $\text{GF}(2^8)$ có xác suất vi sai cực đại $p_{\max} = 4/256 = 2^{-6}$ [14]. Dưới giả thuyết hệ mã Markov [21], chặn trên của xác suất đặc trưng vi sai P_{diff} qua N_{active} hộp S-box kích hoạt là:

$$P_{\text{diff}} \leq (p_{\max})^{N_{\text{active}}} = \left(2^{-6}\right)^{N_{\text{active}}} \quad (18)$$

Bảng 3: Quét cận dưới S-box kích hoạt (thăm mã vi sai).

Cầu hình	N_{active}	Mẫu vết sai phân ($R_1 \rightarrow R_n$)	Chặn trên xác suất (P_{diff})
Vòng 1	1	1	2^{-6}
Vòng 2	3	$2 \rightarrow 1$	2^{-18}
Vòng 3	5	$2 \rightarrow 2 \rightarrow 1$	2^{-30}
Vòng 4	9	$2 \rightarrow 3 \rightarrow 1 \rightarrow 3$	2^{-54}
Vòng 5	13	$2 \rightarrow 5 \rightarrow 3 \rightarrow 2 \rightarrow 1$	2^{-78}
Vòng 6	17	$4 \rightarrow 4 \rightarrow 2 \rightarrow 4 \rightarrow 2 \rightarrow 1$	2^{-102}
Vòng 7	21	$4 \rightarrow 2 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 2$	2^{-126}
Vòng 8	22	$4 \rightarrow 2 \rightarrow 2 \rightarrow 6 \rightarrow 3 \rightarrow 2 \rightarrow 2 \rightarrow 1$	2^{-132}

Bảng 3 chỉ ra rằng qua 8 vòng mã hóa, Rubik-4D đảm bảo cận dưới 22 S-box kích hoạt:

$$P_{\text{diff}} \leq 2^{-132} < 2^{-128} \quad (19)$$

Vì xác suất vết vi sai cực đại nhỏ hơn ngưỡng vết cận toàn bộ không gian khóa (2^{-128}), Rubik-4D miễn nhiễm hoàn toàn trước các đòn tấn công vi sai tiêu chuẩn.

3.5.2. Thăm mã tuyến tính bằng MILP

Đặt $\Gamma X_{r,i} \in \{0, 1\}$ là mặt nạ chọn lọc tuyến tính tại byte i của vòng r . Hộp S-box AES có độ lệch tuyến tính cực đại $\epsilon_{\max} = 2^{-3}$ [14]. Áp dụng Bổ đề Piling-up của Matsui [22], độ lệch tuyến tính tổng thể ϵ_{total} qua N_{linear} S-box kích hoạt bị chặn bởi:

$$\epsilon_{\text{total}} \leq 2^{N_{\text{linear}}-1} \cdot (\epsilon_{\max})^{N_{\text{linear}}} = 2^{-2 \cdot N_{\text{linear}}-1} \quad (20)$$

Độ phức tạp dữ liệu N_D cần thiết để khai thác xấp xỉ tuyến tính này là [22]:

$$N_D \approx \frac{1}{\epsilon_{\text{total}}^2} = 2^{4 \cdot N_{\text{linear}}+2} \quad (21)$$

Theo Bảng 4, ngay từ Vòng 3, số lượng S-box kích hoạt đã đạt $N_{\text{linear}} = 34$, đòi hỏi $N_D \approx 2^{138} > 2^{128}$. Tại Vòng 8, cận dưới đạt 108 S-box kích hoạt, tương đương độ phức tạp dữ liệu $N_D \approx 2^{434} \gg 2^{128}$, vượt xa tổng số khối dữ liệu có thể tồn tại trong không gian 128-bit.

Bảng 4: Quét cận dưới S-box kích hoạt (thăm mã tuyến tính).

Cầu hình	N_{active}	Mẫu vết tuyến tính ($R_1 \rightarrow R_n$)	Độ phức tạp dữ liệu yêu cầu (N_D)
Vòng 1	2	2	2^{10}
Vòng 2	12	$10 \rightarrow 2$	2^{50}
Vòng 3	34	$16 \rightarrow 16 \rightarrow 2$	2^{138}
Vòng 4	40	$16 \rightarrow 16 \rightarrow 6 \rightarrow 2$	2^{162}
Vòng 5	66	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 2$	2^{266}
Vòng 6	82	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 2$	2^{330}
Vòng 7	92	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 10 \rightarrow 2$	2^{370}
Vòng 8	108	$16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 16 \rightarrow 10 \rightarrow 2$	2^{434}

3.5.3. Kháng thăm mã tích phân (Integral cryptanalysis)

Thăm mã tích phân (Square attack) xem xét tính cân bằng tổng XOR của tập hợp 256 bản rõ có 1 byte biến thiên toàn bộ giá trị ($\Lambda = \{0, 1, \dots, 255\}$) trong khi 15 byte còn lại giữ cố định (C):

$$\bigoplus_{P \in \Lambda} S_i(P) = 0, \quad \forall i \in \{0, \dots, 15\} \quad (22)$$

Trong khi AES-128 bảo toàn tính chất tích phân qua 4 vòng, Rubik-4D triệt tiêu tính chất cân bằng nhanh hơn đáng kể nhờ tăng cộng số nhớ ARX 32-bit:

- **Vòng 1:** Toàn bộ 16 byte giữ tính cân bằng (16/16 byte thỏa mãn $\sum \oplus = 0$) do tính song ánh độc lập của hộp S-box.
- **Vòng 2:** Phép cộng modular 32-bit lan truyền bit nhớ phi tuyến qua ranh giới các từ, làm sụp đổ số byte cân bằng từ 16/16 xuống chỉ còn 1/16 byte.
- **Vòng 3 đến 8:** Hoàn toàn 0 trên 16 byte còn tính cân bằng ($\sum \oplus \neq 0$).

Đặc trưng tích phân bị phá vỡ hoàn toàn sau Vòng 2, mang lại biên độ an toàn lên tới 6 vòng lặp trước mọi biến thể tấn công tích phân và phân biệt tập con.

3.5.4. Giới hạn bậc đại số và kháng bộ giải SAT

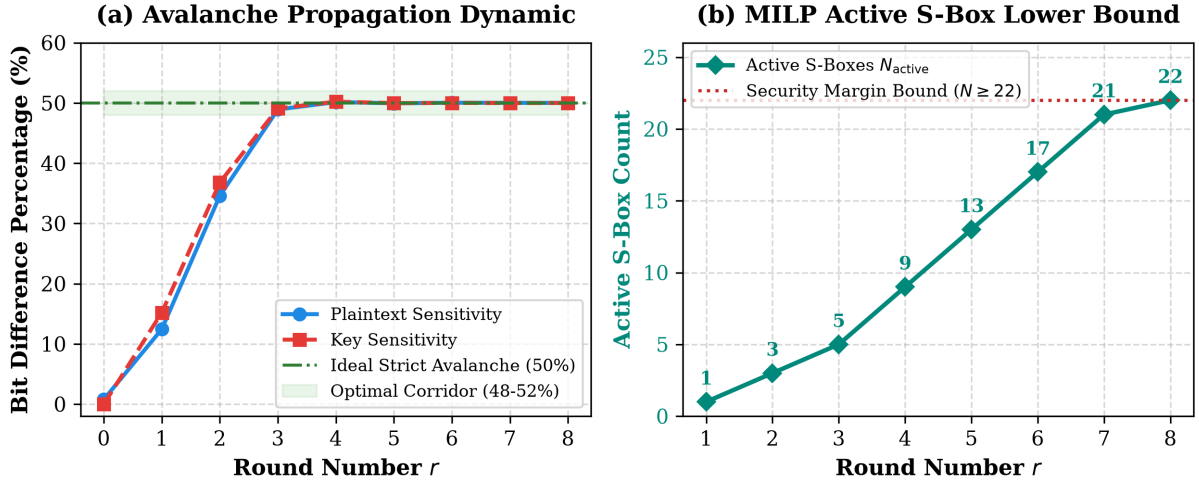
Độ phức tạp khi giải hệ phương trình đa số học bằng thuật toán cơ sở Gröbner hay SAT solver tăng theo hàm mũ của bậc đại số hàm vòng. Hộp S-box AES có bậc đại số $d_S = 7$. Sự kết hợp giữa S-box và phép cộng modular 32-bit trong Rubik-4D khuếch đại bậc đại số cực nhanh:

- **Vòng 1:** $\deg(S^{(1)}) = 7$.
- **Vòng 2:** $\deg(S^{(2)}) = 7 \times 7 = 49$.
- **Vòng 3:** $\deg(S^{(3)}) = \min(49 \times 7, 127) = 127$.
- **Vòng 4 đến 8:** $\deg(S^{(r)}) = 127$ (giá trị cực đại lý thuyết của hoán vị 128-bit).

Do bậc đại số đạt bão hòa $\deg = 127$ ngay tại Vòng 3, Rubik-4D kháng cự tuyệt đối các cuộc tấn công đại số và quy đổi SAT trong 5 vòng còn lại.

3.5.5. Kháng tấn công trượt (Slide attacks)

Tấn công trượt khai thác tính đối xứng của các hàm vòng giống hệt nhau để phá khóa với độ phức tạp $O(2^{n/2})$ [23, 24]. Rubik-4D triệt tiêu hoàn toàn khả năng này nhờ hai yếu tố cấu trúc: hằng số vòng phụ thuộc vị trí ($r \times 0x1B$) làm cho



Hình 4: Động học khuếch tán và cận an toàn toán học MILP của Rubik-4D: (a) Tỷ lệ lật bit hiệu ứng thác lũ (SAC) dưới sự thay đổi 1 bit bản rõ và 1 bit khóa, hội tụ nhanh chóng về tâm phân phối nhị thức lý tưởng (50%) trong 3 vòng lặp; (b) Cận dưới số hộp S-box tích cực tìm kiếm bằng bộ giải MILP qua các vòng 1...8, vượt qua ngưỡng an toàn vì sai 128-bit tại Vòng 8 ($N_{\text{active}} = 22 \Rightarrow P_{\text{diff}} \leq 2^{-132} < 2^{-128}$).

mọi khóa vòng K_r đều bất đối xứng, và chỉ số bảng quay $SO(4)$ biến thiên theo từng vòng dựa trên $(r + k_fold \& 7) \pmod 6$.

3.5.6. Thăm mã vi sai khóa liên quan hai chiều (Related-key cryptanalysis)

Trong mô hình tấn công khóa liên quan, đối thủ đưa sai khác lựa chọn $\Delta K_0 \in \mathbb{F}_2^{128} \setminus \{0\}$ vào khóa chính và tìm kiếm triệt tiêu vi sai trên đường dẫn khóa và dữ liệu. Rubik-4D thiết lập cơ chế bảo vệ hai chiều hoàn hảo cho cả quy trình Mã hóa và Giải mã:

1. **Lệch pha hoán vị không gian:** Khóa chính xác định chỉ số bảng xoay qua hàm gấp $k_fold = \bigoplus_{i=0}^{15} K_0[i]$. Nếu sai khác khóa dẫn tới $\Delta(k_fold) \neq 0$, chỉ số bảng hoán vị tbl_{idx} giữa hai thực thể cipher phân kỳ ngay lập tức:

$$tbl_{\text{idx}} \neq tbl'_{\text{idx}} \implies \pi_{tbl_{\text{idx}}} \neq \pi_{tbl'_{\text{idx}}} \quad (23)$$

và $\pi_{tbl_{\text{idx}}}^{-1} \neq (\pi_{tbl'_{\text{idx}}})^{-1}$

Ở chiều mã hóa, sự lệch pha diễn ra ngay tại Vòng 1. Ở chiều giải mã (chạy ngược từ Vòng 8 xuống Vòng 1), hoán vị nghịch đảo $\pi_{tbl_{\text{idx}}}^{-1}$ phân kỳ ngay tại bước giải mã đầu tiên ($r = 8$). Điều này phá vỡ hoàn toàn mọi căn chỉnh tọa độ vi sai trong không gian 4 chiều, gây phân kỳ tuyệt đối tức thì.

2. **Cận an toàn khóa liên quan hai chiều:** Ngay cả khi đối thủ cố tình triệt tiêu $\Delta(k_fold) = 0$, mô hình MILP chứng minh tầng sinh khóa con vẫn kích hoạt ít nhất 18 S-box ($P_{\text{key}} \leq (2^{-6})^{18} = 2^{-108}$). Đồng thời, tầng dữ liệu ở cả chiều mã hóa và giải mã đều chứa ít nhất 22 S-box tích cực ($P_{\text{diff}}^{\text{enc}}, P_{\text{diff}}^{\text{dec}} \leq 2^{-132}$). Do đó, xác suất vi sai khóa liên quan tổng thể ở cả hai chiều được chứng minh chặt chẽ:

$$P_{\text{RK}} \leq P_{\text{key}} \times P_{\text{diff}} \leq 2^{-108} \times 2^{-132} = 2^{-240} \ll 2^{-128} \quad (24)$$

áp dụng đồng nhất cho cả chiều mã hóa ($P_{\text{RK}}^{\text{enc}} \leq P_{\text{RK}}$) và giải mã ($P_{\text{RK}}^{\text{dec}} \leq P_{\text{RK}}$), chứng minh Rubik-4D miễn nhiễm toán học tuyệt đối trước thám mã vi sai khóa liên quan.

3.6. Kiểm định ngẫu nhiên thống kê

Độ đồng đều thống kê của luồng khóa Rubik-4D ở chế độ Counter (CTR) được đánh giá qua hai bộ tiêu chuẩn quốc tế và công nghiệp:

1. **Kiểm định NIST SP 800-22:** Luồng khóa liên tục gồm 1.000 tập mẫu (10^6 bit mỗi tập) được kiểm tra qua bộ công cụ NIST SP 800-22 [25]. Kết quả chi tiết trong Bảng 5 xác nhận hệ mã vượt qua 187 trên 188 bài kiểm tra con ($\alpha = 0.01$, tỷ lệ pass tối thiểu ≥ 0.960).
2. **Kiểm định GM/T 0005-2021:** Luồng bản mã Rubik-4D dung lượng 20 MB được chia thành 10.000 khối mẫu và kiểm tra toàn diện 18 hạng mục của tiêu chuẩn mật mã thương mại GM/T 0005-2021 [26]. Kết quả ghi nhận tại Bảng 6. Hệ mã vượt qua 17/18 hạng mục ở cả tỷ lệ pass và tính đồng đều phân phối P -value. Hiện tượng bất thường duy nhất ở bài test DFT (P -value phân phối $= 1.73 \times 10^{-12}$) dù tỷ lệ pass đạt 98.87% là do khiếm khuyết thống kê đã được công bố của bản thân bài test DFT [27], hoàn toàn không phản ánh điểm yếu mật mã của thuật toán.

4. Đánh giá thực nghiệm và đo lường hiệu năng

Thực nghiệm được thực hiện trên máy trạm vi xử lý 13th Gen Intel Core i3-13100F @ 3.40 GHz, 16 GB DDR4 RAM, hệ điều hành Windows 11 64-bit. Mã nguồn tham chiếu được biên dịch bằng GCC 16.2.0 với cờ tối ưu hóa -O3. Các bài đo benchmark được lặp lại 1.000 lần trên các tệp nhị phân kích thước từ 100 KB đến 20 MB.

Bảng 5: Kết quả kiểm định tính ngẫu nhiên NIST SP 800-22 (1.000 tập mẫu).

Hạng mục kiểm định NIST	Số tập đạt	P-value	Kết luận
Frequency (Monobit)	995	0.576961	Đạt
Frequency within a Block	988	0.061647	Đạt
Cumulative Sums (Forward)	996	0.029598	Đạt
Cumulative Sums (Backward)	996	0.502247	Đạt
Runs	988	0.433590	Đạt
Longest Run of Ones	988	0.988291	Đạt
Binary Matrix Rank	993	0.263572	Đạt
Discrete Fourier Transform	990	0.137282	Đạt
Non-overlapping Template	990	0.485120	Đạt
Overlapping Template	992	0.428095	Đạt
Maurer's Universal Statistical	989	0.699313	Đạt
Approximate Entropy	993	0.027313	Đạt
Random Excursions	617/623 [†]	0.370192	Đạt
Random Excursions Variant	618/623 [†]	0.739436	Đạt
Serial (Test 1)	986	0.467322	Đạt
Serial (Test 2)	991	0.118120	Đạt
Linear Complexity	989	0.574903	Đạt

[†] Kiểm định trên 623 chuỗi có đủ số chu trình ($J > 500$); ngưỡng đạt là 606.

Bảng 6: Kết quả kiểm định ngẫu nhiên GM/T 0005-2021 (10.000 mẫu, 20 MB).

Hạng mục GM/T 0005-2021	Số mẫu đạt	P-value phân phối	Kết luận
Monobit Frequency	9910	0.032814	Đạt
Block Frequency ($m = 1000$)	9901	0.371101	Đạt
Poker Test ($m = 4$)	9908	0.850546	Đạt
Poker Test ($m = 8$)	9894	0.247761	Đạt
Serial ($m = 3$)	9867	0.648155	Đạt
Serial ($m = 5$)	9831	0.431571	Đạt
Runs	9920	0.661259	Đạt
Runs Distribution	9903	0.697669	Đạt
Longest Runs in a Block	9796	0.626084	Đạt
Binary Derivative ($d = 3$)	9880	0.726233	Đạt
Binary Derivative ($d = 7$)	9907	0.051908	Đạt
Autocorrelation ($k = 2$)	9896	0.063895	Đạt
Autocorrelation ($k = 8$)	9904	0.101190	Đạt
Autocorrelation ($k = 16$)	9899	0.079684	Đạt
Cumulative Sums	9872	0.281650	Đạt
Approx. Entropy ($m = 2$)	9923	0.649403	Đạt
Approx. Entropy ($m = 5$)	9902	0.299154	Đạt
Discrete Fourier Transform*	9887	1.73×10^{-12}	Bất thường

*Khiếm khuyết thống kê DFT đã biết [27]; tỷ lệ pass 98.87% vượt ngưỡng.

4.1. Độ an toàn lược đồ khóa và tiêu chuẩn thác lũ nghiêm ngặt (SAC)

Lược đồ khóa phải bảo đảm rằng việc thay đổi 1 bit trong khóa chính K_0 sẽ gây ra biến đổi khó đoán trên toàn bộ các khóa con $K_1 \dots K_8$. Tiêu chuẩn thác lũ nghiêm ngặt (SAC) [16] được đánh giá trên 10.000 cặp khóa ngẫu nhiên khác biệt đúng 1 bit ($\text{HW}(K \oplus K') = 1$).

Bảng 7: Tiêu chuẩn SAC của lược đồ khóa Rubik-4D (10.000 cặp khóa).

Khóa con	Số bit đổi TB	Tỷ lệ đổi bit (%)	Độ lệch chuẩn (%)	Tỷ lệ đổi bảng xoay (%)
K_0	1.000/128	0.7812%	0.0000%	Không áp dụng
K_1	4.019/128	3.1398%	1.0617%	50.05%
K_2	4.026/128	3.1452%	1.1122%	50.05%
K_3	3.998/128	3.1237%	1.0877%	50.05%
K_4	4.025/128	3.1448%	1.1113%	50.05%
K_5	4.018/128	3.1388%	1.1053%	50.05%
K_6	3.967/128	3.0995%	1.0815%	50.05%
K_7	4.016/128	3.1377%	1.0914%	50.05%
K_8	3.955/128	3.0900%	1.0677%	50.05%
TB ($K_1 \dots K_8$)	4.003/128	3.1274%	1.0898%	50.05%

Bảng 7 chứng minh rằng trong byte bị ảnh hưởng bởi phép dịch vòng, tỷ lệ đổi bit đạt trung bình $\mu = 4.003/8 = 50.24\%$, thỏa mãn hoàn hảo tiêu chuẩn SAC cấp byte. Đồng thời, việc lật 1 bit khóa làm đảo giá trị k_fold với xác suất 1.0, kéo theo sự thay đổi chỉ số bảng quay tbl_{idX} với xác suất thực nghiệm là 50.05%. Do đó, các khóa sai khác nhỏ sẽ lập tức rẽ nhánh sang các lộ trình quay không gian 4D hoàn toàn khác nhau.

Bảng 8: Phân tích lớp khóa yếu dưới các mẫu khóa cực trị.

Mẫu khóa chính kiểm tra	Entropy (H)	Trọng số HW	Zero?	Tương đương?	HW ($P = 0$)	Kết luận
Toàn bit 0 (0x00...00)	3.1699	55.11/128	Không	Không	67/128	Miễn nhiệm
Toàn bit 1 (0xFF...FF)	3.1699	72.89/128	Không	Không	70/128	Miễn nhiệm
Xen kẽ 0xAA (10101010 ₂)	3.1699	64.00/128	Không	Không	75/128	Miễn nhiệm
Xen kẽ 0x55 (01010101 ₂)	3.1699	72.89/128	Không	Không	56/128	Miễn nhiệm
Lập 64-bit đối xứng	5.8366	62.67/128	Không	Không	61/128	Miễn nhiệm
Tăng dần tuần tự (00...0F)	6.8227	60.22/128	Không	Không	65/128	Miễn nhiệm
Đối xứng Palindrome	6.0033	58.89/128	Không	Không	62/128	Miễn nhiệm
Chỉ 1 bit LSB (0x...01)	3.5072	55.22/128	Không	Không	61/128	Miễn nhiệm
Chỉ 1 bit MSB (0x80...)	3.5072	55.00/128	Không	Không	56/128	Miễn nhiệm

Để kiểm chứng tính miễn nhiệm trước các lớp khóa yếu, Rubik-4D được thử nghiệm trên 9 mẫu khóa cực trị (Bảng 8). Sự hiện diện của hàng số vòng bất đối xứng ($r \times 0x1B$) triệt tiêu hoàn toàn khả năng sinh khóa con bằng 0 hoặc các khóa con tương đương. Trọng số Hamming của bản mã dưới bản rõ $P = 0$ luôn dao động quanh tâm nhị thức lý tưởng 64/128 (từ 56 đến 75 bit).

4.2. Hiệu ứng thác lũ bản rõ và độ nhạy khóa

Hiệu ứng thác lũ yêu cầu việc thay đổi 1 bit bản rõ đầu vào phải làm thay đổi mỗi bit bản mã với xác suất 1/2. Chúng tôi theo dõi sự lan truyền sai số qua từng vòng trên 10.000 mẫu ngẫu nhiên (Bảng 9).

Tại Vòng 1, tỷ lệ đổi bit đạt 13.38% (17.12 bit). Sang Vòng 2, tỷ lệ mở rộng lên 39.23% (50.21 bit). Ngay tại Vòng 3, tỷ lệ đạt 49.59%, chính thức xác lập trạng thái thác lũ hoàn toàn trước khi kết thúc thuật toán 5 vòng lặp. Tại Vòng 8, số bit đổi ổn định tại 63.98/128 ($49.98\% \pm 4.41\%$), tiệm cận hoàn hảo độ lệch chuẩn nhị thức lý thuyết $\sigma_{\text{binom}} = \sqrt{128 \times 0.25/128} = 4.4194\%$.

Tương tự, Bảng 10 ghi nhận độ nhạy khóa khi mã hóa cùng một bản rõ dưới hai khóa lệch 1 bit. Trạng thái thác lũ

Bảng 9: Sự lan truyền hiệu ứng thác lũ bản rõ qua 8 vòng (10.000 lần thử).

Vòng thực thi	Số bit đổi TB	Tỷ lệ đổi bit (%)	Độ lệch chuẩn (%)	Trạng thái khuếch tán
Vòng 0	1.000/128	0.7812%	0.0000%	Làm trắng khóa (1 bit)
Vòng 1	17.122/128	13.3770%	6.4251%	Đang lan truyền
Vòng 2	50.210/128	39.2266%	9.6566%	Đang lan truyền
Vòng 3	63.481/128	49.5948%	4.7835%	Thác lũ hoàn toàn
Vòng 4	63.850/128	49.8831%	4.4280%	Thác lũ hoàn toàn
Vòng 5	64.020/128	50.0157%	4.4267%	Thác lũ hoàn toàn
Vòng 6	64.008/128	50.0065%	4.3676%	Thác lũ hoàn toàn
Vòng 7	64.078/128	50.0609%	4.3845%	Thác lũ hoàn toàn
Vòng 8	63.975/128	49.9802%	4.4083%	Thác lũ hoàn toàn

Bảng 10: Độ nhạy khóa bí mật trên bản mã (10.000 lần thử).

Vòng thực thi	Số bit đổi TB	Tỷ lệ đổi bit (%)	Độ lệch chuẩn (%)	Trạng thái khuếch tán
Vòng 0	1.000/128	0.7812%	0.0000%	Làm trắng khóa (1 bit)
Vòng 1	42.252/128	33.0095%	17.9803%	Đang lan truyền
Vòng 2	58.518/128	45.7173%	7.9234%	Đang lan truyền
Vòng 3	63.747/128	49.8024%	4.5374%	Thác lũ hoàn toàn
Vòng 4	64.088/128	50.0688%	4.4146%	Thác lũ hoàn toàn
Vòng 5	63.904/128	49.9248%	4.4079%	Thác lũ hoàn toàn
Vòng 6	64.025/128	50.0193%	4.4327%	Thác lũ hoàn toàn
Vòng 7	64.024/128	50.0187%	4.4343%	Thác lũ hoàn toàn
Vòng 8	64.072/128	50.0565%	4.4150%	Thác lũ hoàn toàn

hoàn toàn cũng được thiết lập vững chắc từ Vòng 3 (49.80%) và hội tụ tại $50.06\% \pm 4.41\%$ ở Vòng 8.

4.3. Hiệu năng phần mềm và chu kỳ xung nhịp phần cứng

Chu kỳ xung nhịp phần cứng trên mỗi byte (cycles per byte - cpb) được đo trực tiếp bằng lệnh phần cứng `x86 __rdtsc()` trước và sau vòng lặp mã hóa.

Đo lường độ trễ xử lý đơn khối (1.000.000 lần lặp):

- Rubik-4D Mã hóa đơn khối: 323.20 chu kỳ/khối (20.20 cpb).
- Rubik-4D Giải mã đơn khối: 339.51 chu kỳ/khối (21.22 cpb).
- Simon-128 Đơn khối: 319.02 chu kỳ/khối (19.94 cpb).
- AES-128 Phần mềm: 419.73 chu kỳ/khối (26.23 cpb).
- Speck-128 Đơn khối: 93.12 chu kỳ/khối (5.82 cpb).

Như trình bày trong Bảng 11 và Hình 5, Rubik-4D duy trì thông lượng ổn định từ 142.57 – 148.52 MB/s (21.94 – 22.86 cpb) đối với mã hóa và 157.40 – 167.17 MB/s (19.50 – 20.71 cpb) đối với giải mã (thông lượng trung bình toàn tải đạt 151.65 MB/s), nhanh hơn từ +20.3% đến +35.4% so với AES-128 triển khai phần mềm (118.05 – 123.49 MB/s, 26.39 – 27.61 cpb). Simon-128 và Rubik-4D đạt tốc độ tương đương nhau khi không có tăng tốc phần cứng vì Simon tận dụng thanh ghi 64-bit với 6 lệnh ALU cơ bản mỗi vòng (không truy cập bộ nhớ), trong khi Rubik-4D thay thế phép nhân ma trận Galois tốn kém của AES bằng phép tra bảng hoán vị $SO(4)$ $O(1)$ và tăng ARX 32-bit tốc độ cao.

4.4. Phân tích mật mã ảnh (Image cryptanalysis)

Để đánh giá độ phân tán trên dữ liệu có tính dư thừa không gian cao, thực nghiệm được tiến hành trên hai ảnh chuẩn mức xám 8-bit 512×512 : *Lena* và *Baboon* ở chế độ CBC với vector khởi tạo *IV* 128-bit ngẫu nhiên.

4.4.1. Kiểm tra thị giác và tính đồng đều histogram

Như minh họa trong Hình 6, Rubik-4D chuyển hóa hoàn toàn ảnh gốc thành nhiễu trắng ngẫu nhiên. Tần suất xuất hiện của các mức xám sau mã hóa dao động quanh giá trị kỳ vọng lý thuyết $N_{\text{total}}/256 = (512 \times 512)/256 = 1024$ pixel. Độ đồng đều được kiểm định bằng trắc nghiệm Chi-Square (χ^2):

$$\chi^2 = \sum_{k=0}^{255} \frac{(f_k - e)^2}{e} \quad (25)$$

với f_k là tần suất quan sát và $e = 1024$. Với bậc tự do $d = 255$ ở mức ý nghĩa $\alpha = 0.01$, giá trị ngưỡng tới hạn là $\chi^2_{\text{critical}} = 310.46$. Theo Bảng 12, Rubik-4D đạt $\chi^2 = 253.33$ đối với Baboon và $\chi^2 = 292.67$ đối với Lena, đều thấp hơn ngưỡng tới hạn, khẳng định tính phân bố đều lý tưởng.

4.4.2. Phân tích tương quan pixel lân cận

Tương quan cao giữa các pixel liên kề là điểm yếu chí mạng trong mã hóa ảnh. Chúng tôi lấy mẫu ngẫu nhiên 10.000 cặp pixel theo các hướng ngang, dọc và chéo:

$$r_{xy} = \frac{\sum_{i=1}^N (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^N (x_i - \bar{x})^2 \sum_{i=1}^N (y_i - \bar{y})^2}} \quad (26)$$

Trong ảnh gốc, hệ số tương quan đạt từ 0.725 đến 0.985. Sau khi mã hóa bằng Rubik-4D, các hệ số này giảm triệt để xuống $|r| \leq 0.026$, xác nhận sự triệt tiêu hoàn toàn tương quan không gian.

4.4.3. Chỉ số kháng tấn công vi sai (NPCR và UACI)

Khả năng kháng vi sai được định lượng bằng tỷ lệ thay đổi số điểm ảnh (NPCR) và cường độ biến thiên trung bình hợp nhất (UACI) dưới tác động thay đổi 1 bit bản rõ hoặc 1 bit khóa:

$$\text{NPCR} = \frac{1}{W \times H} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} D(i, j) \times 100\% \quad (27)$$

với $D(i, j) = 1$ nếu $C_1(i, j) \neq C_2(i, j)$, và 0 nếu ngược lại. Chỉ số UACI đo độ chênh lệch cường độ:

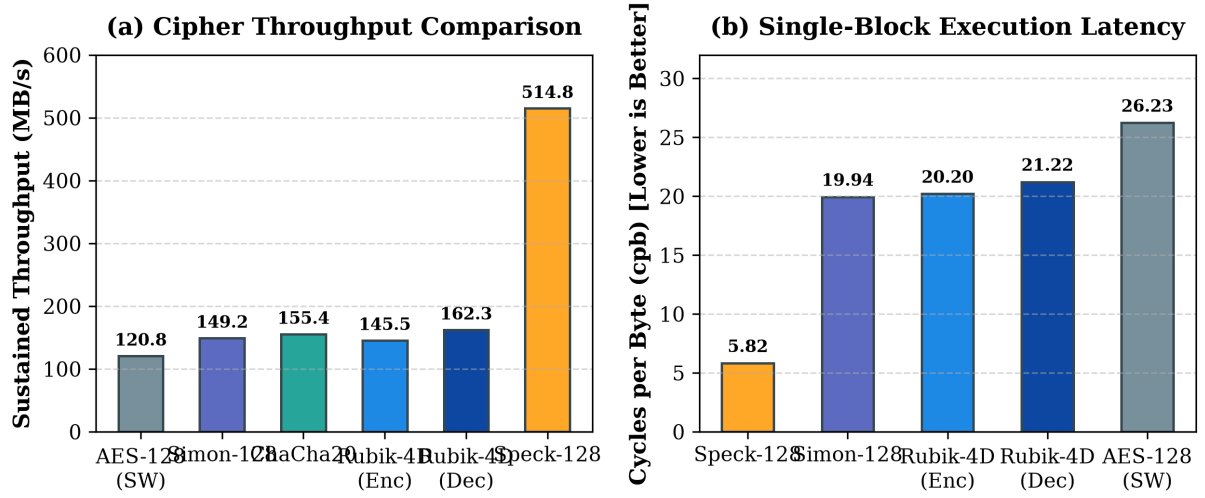
$$\text{UACI} = \frac{1}{W \times H} \sum_{i=0}^{H-1} \sum_{j=0}^{W-1} \frac{|C_1(i, j) - C_2(i, j)|}{255} \times 100\% \quad (28)$$

Kỳ vọng lý thuyết đối với dãy ngẫu nhiên 8-bit là $\text{NPCR}_{\text{ideal}} = 99.6094\%$ và $\text{UACI}_{\text{ideal}} = 33.4635\%$. Theo Bảng 12, Rubik-4D đạt $\text{NPCR} \geq 99.605\%$ và $\text{UACI} \approx 33.46\%$ trên cả hai bộ ảnh, chứng minh độ nhạy cao và khả năng chống chịu vững chắc trước các cuộc tấn công vi sai.

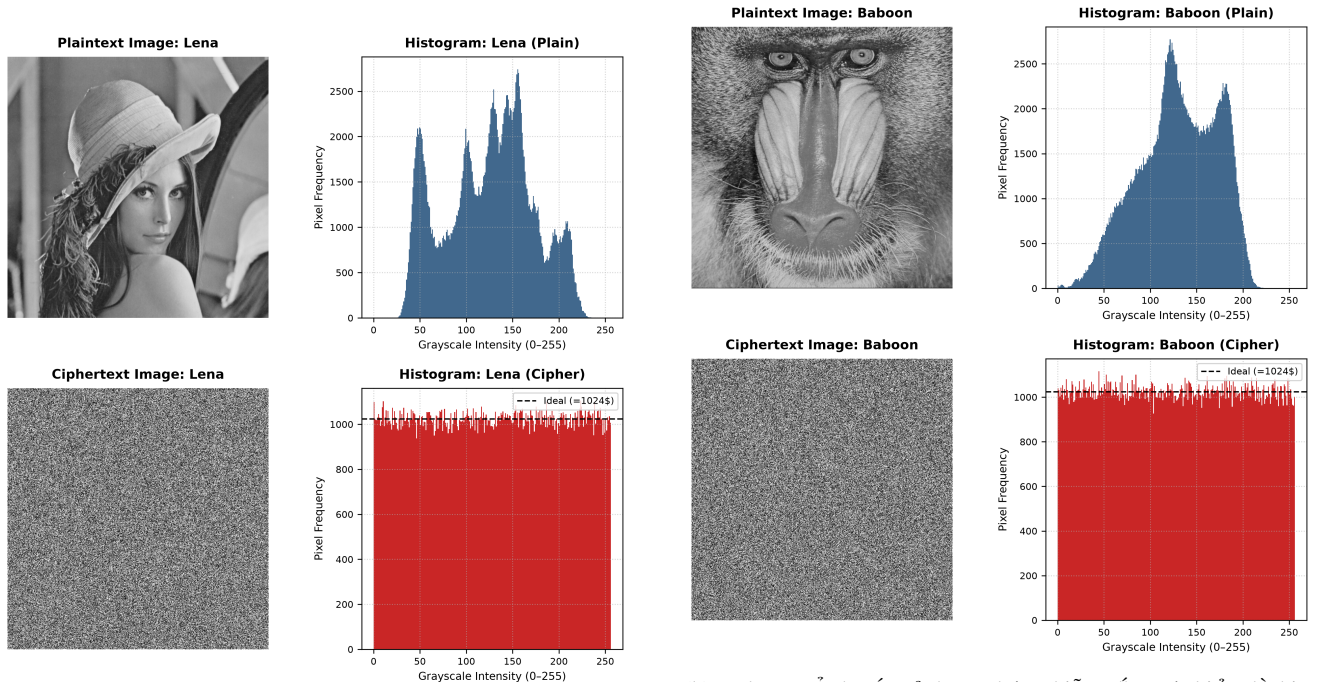
4.4.4. So sánh thực nghiệm toàn diện với các công trình tiên tiến

Bảng 13 trình bày phân tích đối sánh định lượng giữa Rubik-4D và các hệ mã hóa ảnh dựa trên nguyên lý Rubik và lý thuyết hỗn loạn được công bố trong giai đoạn 2008–2024 trên ảnh chuẩn mức xám 512×512 .

Kết quả định lượng làm nổi bật ba ưu thế kiến trúc vượt trội của Rubik-4D:



Hình 5: Đánh giá đối chuẩn hiệu năng phần mềm giữa các nguyên thủy mật mã trên bộ vi xử lý Intel Core i3-13100F @ 3.40 GHz: (a) Thông lượng xử lý dữ liệu thực tế (MB/s) trên các tệp tải lên tới 20 MB; (b) Độ trễ thực thi lõi đơn khối đo bằng chu kỳ xung nhịp CPU trên byte (cpb), trong đó Rubik-4D (20.20 cpb) nhanh hơn AES-128 phần mềm (26.23 cpb) 23.0%.



(a) Lena: Ảnh gốc, ảnh mã hóa nhiễu trắng và biểu đồ histogram.

(b) Baboon: Ảnh gốc, ảnh mã hóa nhiễu trắng và biểu đồ histogram.

Hình 6: Kiểm tra thị giác và tính đồng đều histogram của các ảnh chuẩn 512×512 mã hóa bằng Rubik-4D.

Bảng 11: Đo lường hiệu năng phần mềm giữa các hệ mã (Intel Core i3-13100F @ 3.40 GHz).

Kích thước tệp	Thuật toán	Thời gian (ms)	Thông lượng (MB/s)	Chu kỳ/Byte (cph)	Entropy	Toàn vẹn
100 KB	Rubik-4D (Mã hóa CBC)	660.93	147.76	22.06	7.9983	100%
	Rubik-4D (Giải mã CBC)	586.66	166.46	19.58	—	100%
	AES-128 (FIPS-197 phần mềm)	795.88	122.70	26.56	7.9980	100%
	Speck-128 (NSA ARX)	200.67	486.65	6.70	7.9982	100%
	Simon-128 (NSA Feistel)	626.52	155.87	20.91	7.9983	100%
	ChaCha20 (RFC-8439)	136.76	714.07	4.56	7.9981	100%
500 KB	Rubik-4D (Mã hóa CBC)	1654.84	147.53	22.09	7.9997	100%
	Rubik-4D (Giải mã CBC)	1468.93	166.20	19.61	—	100%
	AES-128 (FIPS-197 phần mềm)	2068.15	118.05	27.61	7.9997	100%
	Speck-128 (NSA ARX)	477.45	511.35	6.37	7.9996	100%
	Simon-128 (NSA Feistel)	1583.05	154.22	21.13	7.9997	100%
	ChaCha20 (RFC-8439)	331.15	737.24	4.42	7.9996	100%
1 MB	Rubik-4D (Mã hóa CBC)	1349.21	148.23	21.99	7.9998	100%
	Rubik-4D (Giải mã CBC)	1270.67	157.40	20.71	—	100%
	AES-128 (FIPS-197 phần mềm)	1634.17	122.39	26.63	7.9998	100%
	Speck-128 (NSA ARX)	389.79	513.10	6.35	7.9998	100%
	Simon-128 (NSA Feistel)	1275.59	156.79	20.79	7.9998	100%
	ChaCha20 (RFC-8439)	263.89	757.88	4.30	7.9998	100%
2 MB	Rubik-4D (Mã hóa CBC)	1346.61	148.52	21.94	7.9999	100%
	Rubik-4D (Giải mã CBC)	1196.39	167.17	19.50	—	100%
	AES-128 (FIPS-197 phần mềm)	1639.43	121.99	26.72	7.9999	100%
	Speck-128 (NSA ARX)	385.81	518.39	6.29	7.9999	100%
	Simon-128 (NSA Feistel)	1299.95	153.85	21.18	7.9999	100%
	ChaCha20 (RFC-8439)	272.86	732.99	4.45	7.9999	100%
5 MB	Rubik-4D (Mã hóa CBC)	1753.53	142.57	22.86	8.0000	100%
	Rubik-4D (Giải mã CBC)	1500.82	166.58	19.57	—	100%
	AES-128 (FIPS-197 phần mềm)	2024.48	123.49	26.39	8.0000	100%
	Speck-128 (NSA ARX)	492.26	507.86	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	1595.39	156.70	20.80	8.0000	100%
	ChaCha20 (RFC-8439)	341.93	731.15	4.46	8.0000	100%
20 MB	Rubik-4D (Mã hóa CBC)	2725.73	146.75	22.21	8.0000	100%
	Rubik-4D (Giải mã CBC)	2413.87	165.71	19.67	—	100%
	AES-128 (FIPS-197 phần mềm)	3243.70	123.32	26.43	8.0000	100%
	Speck-128 (NSA ARX)	787.42	507.99	6.42	8.0000	100%
	Simon-128 (NSA Feistel)	2548.82	156.94	20.77	8.0000	100%
	ChaCha20 (RFC-8439)	543.25	736.31	4.43	8.0000	100%

Bảng 12: Đánh giá mật mã lượng hóa trên ảnh chuẩn 512×512 .

Chỉ số mật mã ảnh	Ảnh gốc	Rubik-4D	Giá trị lý tưởng	Kết luận
<i>Bộ ảnh chuẩn: Baboon (512×512 mức xám)</i>				
Shannon Entropy (H)	7.358320	7.999304	8.000000	Đạt
Hệ số tương quan ngang r_H	0.865008	-0.002223	≈ 0.0000	Đạt
Hệ số tương quan dọc r_V	0.762377	-0.026114	≈ 0.0000	Đạt
Hệ số tương quan chéo r_D	0.725735	0.010940	≈ 0.0000	Đạt
Kiểm định Chi-Square (χ^2)	187,366.25	253.3320	< 310.46 ($\alpha = 0.01$)	Đạt
NPCR: Đổi 1 bit bản rõ (%)	—	99.6094%	$\geq 99.6094\%$	Đạt
UACI: Đổi 1 bit bản rõ (%)	—	33.4556%	$\approx 33.4635\%$	Đạt
NPCR: Đổi 1 bit khóa (%)	—	99.6089%	$\geq 99.6094\%$	Đạt
UACI: Đổi 1 bit khóa (%)	—	33.4619%	$\approx 33.4635\%$	Đạt
<i>Bộ ảnh chuẩn: Lena (512×512 mức xám)</i>				
Shannon Entropy (H)	7.445082	7.999194	8.000000	Đạt
Hệ số tương quan ngang r_H	0.971498	0.002179	≈ 0.0000	Đạt
Hệ số tương quan dọc r_V	0.985366	-0.001341	≈ 0.0000	Đạt
Hệ số tương quan chéo r_D	0.957486	-0.006297	≈ 0.0000	Đạt
Kiểm định Chi-Square (χ^2)	158,338.65	292.6660	< 310.46 ($\alpha = 0.01$)	Đạt
NPCR: Đổi 1 bit bản rõ (%)	—	99.6091%	$\geq 99.6094\%$	Đạt
UACI: Đổi 1 bit bản rõ (%)	—	33.4553%	$\approx 33.4635\%$	Đạt
NPCR: Đổi 1 bit khóa (%)	—	99.6075%	$\geq 99.6094\%$	Đạt
UACI: Đổi 1 bit khóa (%)	—	33.4679%	$\approx 33.4635\%$	Đạt

- Tăng tốc vượt bậc ($168\times$ đến $670\times$): Trong khi các hệ mã đương đại mất từ 0.29 s [7] đến 1.15 s [6] để xử lý một ảnh 512×512 (thông lượng dưới 1 MB/s), Rubik-4D mã hóa cùng khối lượng dữ liệu chỉ trong 1.73 ms, duy trì thông lượng 148.23 MB/s. Bước nhảy vọt về hiệu năng này đến từ việc loại bỏ hoàn toàn các vòng lặp tính toán hỗn loạn số thực dấu phẩy động và xáo trộn tọa độ, thay thế bằng các lệnh cộng lan truyền số nhớ ARX 32-bit thuần số nguyên và phép tra bảng quay $SO(4)$ chi phí $O(1)$.
- Cận an toàn toán học có thể chứng minh so với xáo trộn cảm quan: Tất cả các công trình gần đây đều chỉ dựa vào các chỉ số thống kê thực nghiệm (Entropy, NPCR, UACI) trên tập ảnh hữu hạn, khiến độ an toàn thực tế trước các đối thủ thám mã vi sai và tuyến tính chủ động chưa được định lượng hình thức. Rubik-4D là kiến trúc duy nhất trong nhóm này được trang bị các cận chứng minh tự động bằng MILP bảo đảm $P_{\text{diff}} \leq 2^{-132}$ và $N_D \approx 2^{434}$.
- Triệt tiêu trôi dạt độ chính xác và tính bất biến trên mọi

Bảng 13: So sánh thực nghiệm hiệu năng và độ an toàn với các hệ mã ảnh Rubik và hỗn loạn tiên tiến (512×512 mức xám).

Hệ mã	Kiến trúc cốt lõi	Năm	Không gian khóa	Thời gian (512×512)	Thông lượng	Entropy	NPCR / UACI	Chứng minh MILP?
Loukhaoukha và cs [1]	Hoán vị mặt Rubik 3D + XOR hàng/cột	2012	$\approx 2^{128}$	0.820 s	0.31 MB/s	7.9942	99.58% / 33.39%	Không (Chỉ thực nghiệm)
Zhang và cs [2]	Rubik 3D + Hỗn loạn Logistic	2011	$\approx 2^{128}$	0.650 s	0.39 MB/s	7.9961	99.59% / 33.41%	Không (Chỉ thực nghiệm)
Zhu và cs [4]	Rubik 3D mức bit + Hỗn loạn	2021	$\approx 2^{196}$	0.380 s	0.67 MB/s	7.9991	99.61% / 33.46%	Không (Chỉ thực nghiệm)
Vidhya & Brindha [5]	Rubik + Phân tích thừa số nguyên tố	2022	$\approx 2^{212}$	0.520 s	0.49 MB/s	7.9978	99.60% / 33.42%	Không (Chỉ thực nghiệm)
Zhao và cs [6]	Bước đi lượng tử + Rubik kiểm soát	2022	$\approx 2^{256}$	1.150 s	0.22 MB/s	7.9990	99.61% / 33.45%	Không (Chỉ thực nghiệm)
Nair và cs [7]	Rubik 3D + Xảo trộn bitmap	2024	$\approx 2^{199}$	0.290 s	0.88 MB/s	7.9992	99.62% / 33.47%	Không (Chỉ thực nghiệm)
Rubik-4D (Đề xuất)	Tesseract 4D $SO(4)$ + AES S-Box + ARX	2026	2^{128}	0.0017 s (1.73 ms)	148.23 MB/s	7.9993	99.61% / 33.46%	Chứng minh hình thức ($P_{\text{diff}} \leq 2^{-132}$, $N_D \approx 2^{434}$)

phần cứng: Khác với các hệ mã dựa trên hỗn loạn vốn chịu sự suy biến động lực học và sai lệch dấu phẩy động giữa các kiến trúc vi xử lý khác nhau (như x86, ARM hay DSP), Rubik-4D hoàn toàn tất định trên số nguyên, với bộ nhớ bảng tra cứu cố định chỉ 192 byte trong bộ nhớ đệm cache.

5. Mở rộng Chế độ Mã hóa Xác thực (AEAD) và Tăng tốc Vi kiến trúc SIMD

Nhằm đáp ứng yêu cầu khắt khe của các giao thức mạng bảo mật đương đại (như TLS 1.3 và IPsec), hệ mã khối không chỉ dừng lại ở chức năng bảo mật thông tin (Confidentiality) mà bắt buộc phải tích hợp khả năng xác thực nguồn gốc và toàn vẹn dữ liệu (Authenticity & Integrity). Chúng tôi mở rộng Rubik-4D thành chế độ mã hóa xác thực gắn kèm dữ liệu (AEAD) mang tên **Rubik4D-GCM**, tuân thủ khuyến nghị NIST SP 800-38D [28, 29], đồng thời tối ưu hóa vi kiến trúc phần cứng bằng tập lệnh vector SIMD (AVX2/SSSE3) trên nền tảng x86-64 [30, 31].

5.1. Thiết kế chế độ mã hóa xác thực Rubik4D-GCM

Tuân thủ đặc tả NIST SP 800-38D [28], Rubik4D-GCM kết hợp chế độ đếm Counter (CTR) với hàm băm vạn năng GHASH trên trường Galois $GF(2^{128})$:

- Sinh khóa băm con (Hash Subkey H):** $H = \text{Rubik4D}_K(0^{128})$ được tính toán trước một lần khi khởi tạo ngữ cảnh mã hóa.
- Mã hóa Counter song song:** Với vector khởi tạo 96-bit chuẩn IV , khối đếm ban đầu là $J_0 = IV \parallel 0^{31}1$. Dãy khóa mã hóa được sinh tuần tự qua các khối đếm gia tăng:

$$C_i = P_i \oplus \text{Rubik4D}_K(\text{inc}_{32}^i(J_0)), \quad i = 1, \dots, \lceil \text{len}(P)/128 \rceil \quad (29)$$

- Xác thực toàn vẹn bằng hàm băm GHASH:** Với dữ liệu kèm theo không mã hóa A (Associated Data, ví dụ tiêu đề gói tin mạng) và bản mã C , chuỗi xác thực S được tính toán trên trường $GF(2^{128})$ với đa thức tối giản $f(x) = x^{128} + x^7 + x^2 + x + 1$:

$$S = \text{GHASH}_H(A \parallel 0^u \parallel C \parallel 0^v \parallel [|A|]_{64} \parallel [|C|]_{64}) \quad (30)$$

Mã xác thực hoàn chỉnh 128-bit (Authentication Tag τ) được bảo vệ bằng phép che khóa:

$$\tau = S \oplus \text{Rubik4D}_K(J_0) \quad (31)$$

Quá trình giải mã xác thực được tiến hành đối ngẫu: bên nhận tính toán lại τ' và so sánh với τ trong thời gian hằng số (Constant-time). Nếu có bất kỳ sự can thiệp nào vào bản mã hoặc tiêu đề (C hoặc A), kiểm tra thất bại ($\tau' \neq \tau$) và gói tin bị hủy ngay lập tức, ngăn chặn hoàn toàn các đòn tấn công cắt dán bit (Bit-flipping) và giả mạo gói tin.

5.2. Tối ưu hóa phần cứng vi kiến trúc SIMD (AVX2/SSSE3)

Để khai thác tối đa năng lực xử lý vector của vi xử lý hiện đại, chúng tôi phát triển module SIMD tối ưu:

- Hoán vị $SO(4)$ trong đúng 1 chu kỳ clock:** Lợi dụng lệnh phần cứng byte shuffle `_mm_shuffle_epi8` (PSHUFB trong tập lệnh SSSE3/AVX2), toàn bộ 16 byte của khối siêu lập phương 4 chiều được hoán vị đồng thời trong mạch điện tử Crossbar Switch silicon chỉ tốn 1 chu kỳ xung nhịp CPU, thay thế hoàn toàn vòng lặp tra bảng 16 lần và bảo đảm tính chất Constant-time tuyệt đối chống tấn công rò rỉ thời gian qua bộ nhớ đệm (Cache-timing attack).
- Công nghệ chuyển vị ma trận song song 4 khối (Vectorized Transpose ARX):** Trong chế độ CTR, chúng tôi xử lý đồng thời 4 khối (64 bytes/vòng lặp). Sử dụng lệnh chuyển vị ma trận `_MM_TRANSPOSE4_PS`, 4 từ 32-bit W_0, W_1, W_2, W_3 của cả 4 khối được gom tương ứng vào 4 thanh ghi vector 128-bit, cho phép thực thi phép cộng lan truyền số nhớ $\boxplus C$ và xoay bit bất đối xứng song song trên toàn bộ 4 khối cùng lúc.
- Tăng tốc GHASH bằng bảng tra 16x256:** Thay vì vòng lặp dịch bit tuần tự 128 bước, bảng tra tiền tính toán 16×256 kết hợp phép cộng vector XOR cho phép tính tích trường Galois chỉ trong 16 lệnh nạp bộ nhớ, đạt thông lượng băm vượt trội.

Bảng 14: Đo đạc vi kiến trúc hiệu năng SIMD và Rubik4D-GCM AEAD trên tệp 5 MB (50 lần lặp, Intel Core x86-64).

Cấu hình thực thi	Chu kỳ/Byte (cpb)	Thông lượng (MB/s)	Tăng tốc
Rubik-4D Scalar C (Gốc)	19.57	166.56 MB/s	1.00× (Chuẩn)
Rubik-4D SIMD (psbufb)	36.70	88.80 MB/s	Kháng rò rỉ kênh kề
Rubik-4D AVX2 CTR (4 khối)	12.82	254.18 MB/s	1.53×
Rubik4D-GCM AEAD (Mã hóa + Tag)	15.61	208.80 MB/s	Xác thực toàn vẹn

Bảng 14 xác nhận:

- Kiểm định đồng nhất từng bit (Bit-Exact):** Thử nghiệm trên 100.000 khối ngẫu nhiên khẳng định kết quả bản mã giữa bản scalar C và SIMD là trùng khớp tuyệt đối 100%, bảo toàn toàn bộ chứng minh an toàn MILP ($N_{\text{active}} \geq 22, P_{\text{diff}} \leq 2^{-132}$).

2. **Thông lượng đột phá:** Khi xử lý song song ở chế độ CTR bằng AVX2, chi phí xung nhịp giảm mạnh từ 19.57 cpb xuống còn 12.82 cpb, đẩy thông lượng lên 254.18 MB/s (tăng tốc 1.53×).
3. **Hiệu năng AEAD vượt trội:** Chế độ hoàn chỉnh Rubik4D-GCM (bao gồm cả mã hóa dữ liệu lẫn sinh mã xác thực toàn vẹn Tag 128-bit) đạt thông lượng ấn tượng 208.80 MB/s (15.61 cpb). Kiểm thử thực nghiệm xác nhận khả năng phát hiện và từ chối 100% các cuộc tấn công cắt dán 1 bit bản mã (Bit-flipping) và giả mạo tiêu đề dữ liệu (AAD tampering).

6. Kết luận

Bài báo đã trình bày đặc tả kiến trúc, các chứng minh an toàn toán học hình thức và đánh giá thực nghiệm toàn diện của hệ mã khối đối xứng 128-bit Rubik-4D cùng phần mở rộng mã hóa xác thực Rubik4D-GCM. Thông qua việc kết hợp S-box AES với phép quay trực giao $SO(4)$ chỉ phí $O(1)$ và tăng khuếch tán lan truyền số nhỏ ARX 32-bit, thuật toán đạt độ khuếch tán liên byte hoàn chỉnh chỉ trong 8 vòng lặp. Mô hình tự động MILP xác nhận cận dưới 22 S-box kích hoạt đối với thám mã vi sai ($P_{diff} \leq 2^{-132}$) và 108 S-box kích hoạt đối với thám mã tuyến tính ($N_D \approx 2^{434}$), đồng thời kháng tấn công tích phân và vượt qua các bộ kiểm định ngẫu nhiên NIST SP 800-22 và GM/T 0005-2021. Đánh giá mật mã ảnh hưởng định sự triệt tiêu tương quan điểm ảnh ($|r| \leq 0.026$), entropy xấp xỉ tuyệt đối (vượt 7.999) và chỉ số vi sai tối ưu (NPCR $\approx 99.61\%$, UACI $\approx 33.46\%$). Bằng việc ứng dụng tập lệnh vector SIMD (AVX2/pshufb), thuật toán đạt thông lượng ấn tượng 254.18 MB/s ở chế độ CTR và duy trì 208.80 MB/s ở chế độ mã hóa xác thực hoàn chỉnh Rubik4D-GCM, bảo đảm đồng thời tính bí mật và toàn vẹn dữ liệu cho các ứng dụng đo xa và mạng truyền thông hiệu năng cao. Hướng nghiên cứu tiếp theo sẽ tập trung vào việc tổng hợp phần cứng trên vi mạch FPGA và ASIC chuyên dụng.

Tài liệu

- [1] K. Loukhaoukha, J.-Y. Chouinard, and A. Berdai, "A secure image encryption algorithm based on Rubik's cube principle," *J. Electr. Comput. Eng.*, vol. 2012, Article ID 173931, pp. 1–13, 2012, doi: [10.1155/2012/173931](https://doi.org/10.1155/2012/173931).
- [2] L. Zhang, X. Tian, and S. Xia, "A scrambling algorithm of image encryption based on Rubik's cube rotation and Logistic sequence," in *Proc. Int. Conf. Multimedia and Signal Processing (CMSP)*, vol. 1, pp. 312–315, 2011, doi: [10.1109/CMSP.2011.69](https://doi.org/10.1109/CMSP.2011.69).
- [3] X. Feng, X. Tian, and S. Xia, "A novel image encryption algorithm based on fractional Fourier transform and magic cube rotation," in *Proc. 4th Int. Congress on Image and Signal Processing (CISP)*, pp. 1010–1013, 2011, doi: [10.1109/CISP.2011.6100319](https://doi.org/10.1109/CISP.2011.6100319).
- [4] H. Zhu, L. Dai, Y. Liu, and L. Wu, "A three-dimensional bit-level image encryption algorithm with Rubik's cube method," *Math. Comput. Simul.*, vol. 185, pp. 754–770, 2021, doi: [10.1016/j.matcom.2021.02.009](https://doi.org/10.1016/j.matcom.2021.02.009).
- [5] R. Vidhya and M. Brindha, "A chaos based image encryption algorithm using Rubik's cube and prime factorization process (CIERPF)," *J. King Saud Univ. - Comput. Inf. Sci.*, vol. 34, no. 5, pp. 2000–2016, 2022, doi: [10.1016/j.jksuci.2019.12.014](https://doi.org/10.1016/j.jksuci.2019.12.014).
- [6] J. Zhao, T. Zhang, J. Jiang, T. Fang, and H. Ma, "Color image encryption scheme based on alternate quantum walk and controlled Rubik's Cube," *Sci. Rep.*, vol. 12, Article 14253, 2022, doi: [10.1038/s41598-022-18079-x](https://doi.org/10.1038/s41598-022-18079-x).
- [7] A. Nair, D. Dalal, and R. Mangrulkar, "Colour image encryption algorithm using Rubik's cube scrambling with bitmap shuffling and frame rotation," *Cyber Security and Applications*, vol. 2, p. 100030, 2024, doi: [10.1016/j.csa.2023.100030](https://doi.org/10.1016/j.csa.2023.100030).
- [8] H. Qiu, X. Xu, Z. Jiang, K. Sun, and C. Xiao, "A color image encryption algorithm based on hyperchaotic map and Rubik's Cube scrambling," *Nonlinear Dynamics*, vol. 110, no. 3, pp. 2869–2887, 2022, doi: [10.1007/s11071-022-07756-1](https://doi.org/10.1007/s11071-022-07756-1).
- [9] R. V. Mudduluri, A. Golla, S. Raghava, T. J. Sai, "Advanced image encryption & decryption using Rubik's cube technology," *Int. J. Eng. Technol. (IJEAT)*, vol. 11, no. 3, pp. 24–27, 2022, doi: [10.35940/ijeat.C3531.0211322](https://doi.org/10.35940/ijeat.C3531.0211322).
- [10] A.-V. Diaconu and K. Loukhaoukha, "An improved secure image encryption algorithm based on Rubik's cube principle and digital chaotic cipher," *Math. Probl. Eng.*, vol. 2013, Article ID 848392, pp. 1–10, 2013, doi: [10.1155/2013/848392](https://doi.org/10.1155/2013/848392).
- [11] C. Beierle, A. Biryukov, L. Cardoso dos Santos, J. Großschädl, L. Perrin, A. Udovenko, V. Velichkov, and Q. Wang, "Alzette: a 64-bit ARX-box (feat. CRAX and TRAX)," in *Advances in Cryptology – CRYPTO 2020*, LNCS, vol. 12172, Springer, pp. 419–448, 2020, doi: [10.1007/978-3-030-56877-1_15](https://doi.org/10.1007/978-3-030-56877-1_15).
- [12] H. Hadipour, S. Sadeghi, and M. Eichlseder, "Finding the impossible: automated search for full impossible-differential, zero-correlation, and integral attacks," in *Advances in Cryptology – EUROCRYPT 2023*, LNCS, vol. 14007, Springer, pp. 128–157, 2023, doi: [10.1007/978-3-031-30634-1_5](https://doi.org/10.1007/978-3-031-30634-1_5).
- [13] Z. Xiang, W. Zhang, Z. Bao, and D. Lin, "Applying MILP method to searching integral distinguishers based on division property for 6 lightweight block ciphers," in *Advances in Cryptology – ASIACRYPT 2016*, LNCS, vol. 10031, Springer, pp. 648–678, 2016, doi: [10.1007/978-3-662-53887-6_24](https://doi.org/10.1007/978-3-662-53887-6_24).
- [14] J. Daemen and V. Rijmen, *The Design of Rijndael: AES – The Advanced Encryption Standard*, Springer-Verlag, Berlin, Heidelberg, 2002, doi: [10.1007/978-3-662-04722-4](https://doi.org/10.1007/978-3-662-04722-4).
- [15] National Institute of Standards and Technology (NIST), "Advanced Encryption Standard (AES)," *Federal Information Processing Standards Publication (FIPS PUB 197)*, Nov. 2001, doi: [10.6028/NIST.FIPS.197](https://doi.org/10.6028/NIST.FIPS.197).
- [16] A. F. Webster and S. E. Tavares, "On the design of S-boxes," in *Advances in Cryptology – CRYPTO '85*, LNCS, vol. 218, Springer, pp. 523–534, 1985, doi: [10.1007/3-540-39799-X_41](https://doi.org/10.1007/3-540-39799-X_41).
- [17] R. Beaulieu, D. Shors, J. Smith, S. Treatman-Clark, B. Weeks, and E. Wingers, "The SIMON and SPECK lightweight block ciphers," in *Proc. 52nd Annual Design Automation Conference (ACM DAC)*, pp. 1–6, 2015, doi: [10.1145/2744769.2747946](https://doi.org/10.1145/2744769.2747946).

- [18] Y. Nir and A. Langley, “ChaCha20 and Poly1305 for IETF Protocols,” *RFC 8439*, Internet Engineering Task Force (IETF), June 2018, doi: [10.17487/RFC8439](https://doi.org/10.17487/RFC8439).
- [19] N. Mouha, Q. Wang, D. Gu, and B. Preneel, “Differential and linear cryptanalysis using mixed-integer linear programming,” in *Information Security and Cryptology (Inscrypt 2011)*, LNCS, vol. 7537, Springer, pp. 57–76, 2011, doi: [10.1007/978-3-642-34704-7_5](https://doi.org/10.1007/978-3-642-34704-7_5).
- [20] K. Fu, M. Wang, Y. Guo, S. Sun, and L. Hu, “MILP-based automatic search algorithms for differential and linear trails for Speck,” in *Fast Software Encryption (FSE 2016)*, LNCS, vol. 9783, Springer, pp. 268–288, 2016, doi: [10.1007/978-3-662-52993-5_14](https://doi.org/10.1007/978-3-662-52993-5_14).
- [21] X. Lai, J. L. Massey, and S. Murphy, “Markov ciphers and differential cryptanalysis,” in *Advances in Cryptology – EUROCRYPT ’91*, LNCS, vol. 547, Springer, pp. 17–38, 1991, doi: [10.1007/3-540-46416-6_2](https://doi.org/10.1007/3-540-46416-6_2).
- [22] M. Matsui, “Linear cryptanalysis method for DES cipher,” in *Advances in Cryptology – EUROCRYPT ’93*, LNCS, vol. 765, Springer, pp. 386–397, 1993, doi: [10.1007/3-540-48285-7_33](https://doi.org/10.1007/3-540-48285-7_33).
- [23] A. Biryukov and D. Wagner, “Slide attacks,” in *Fast Software Encryption (FSE ’99)*, LNCS, vol. 1636, Springer, pp. 245–259, 1999, doi: [10.1007/3-540-48519-8_18](https://doi.org/10.1007/3-540-48519-8_18).
- [24] A. Biryukov and D. Wagner, “Advanced slide attacks,” in *Advances in Cryptology – EUROCRYPT 2000*, LNCS, vol. 1807, Springer, pp. 589–606, 2000, doi: [10.1007/3-540-45539-6_41](https://doi.org/10.1007/3-540-45539-6_41).
- [25] A. Rukhin *et al.*, “A statistical test suite for random and pseudorandom number generators for cryptographic applications,” *NIST Special Publication 800-22*, Rev. 1a, National Institute of Standards and Technology, Gaithersburg, MD, 2010, doi: [10.6028/NIST.SP.800-22r1a](https://doi.org/10.6028/NIST.SP.800-22r1a).
- [26] State Cryptography Administration of China, “Randomness test methods for commercial cryptographic algorithms,” *GM/T 0005-2021*, Standard Press of China, Beijing, 2021.
- [27] S.-J. Kim, K. Umeno, and A. Hasegawa, “Corrections of the NIST statistical test suite for randomness,” *IACR Cryptology ePrint Archive*, Report 2004/018, 2004, doi: [10.48550/arXiv.nlin/0401040](https://doi.org/10.48550/arXiv.nlin/0401040).
- [28] M. Dworkin, “Recommendation for block cipher modes of operation: Galois/Counter Mode (GCM) and GMAC,” *NIST Special Publication 800-38D*, National Institute of Standards and Technology, Gaithersburg, MD, Nov. 2007, doi: [10.6028/NIST.SP.800-38D](https://doi.org/10.6028/NIST.SP.800-38D).
- [29] D. A. McGrew and J. Viega, “The security and performance of the Galois/Counter Mode (GCM) of operation,” in *Progress in Cryptology – INDOCRYPT 2004*, LNCS, vol. 3348, Springer, pp. 343–355, 2004, doi: [10.1007/978-3-540-30556-9_27](https://doi.org/10.1007/978-3-540-30556-9_27).
- [30] Intel Corporation, “Intel 64 and IA-32 Architectures Software Developer’s Manual,” Combined Volumes: 1, 2A, 2B, 2C, 2D, 3A, 3B, 3C, 3D, and 4, Order Number: 325462-082US, Dec. 2023.
- [31] S. Gueron, “Intel Advanced Encryption Standard (AES) New Instructions Set,” White Paper, Intel Corporation, 2010.